



中国研究生创新实践系列大赛
中国光谷·“华为杯”第十九届中国研究生
数学建模竞赛

学 校 东南大学

参赛队号 22102860210

	1.陈雨荷
队员姓名	2.王牵莲
	3.朱弘智

中国研究生创新实践系列大赛

中国光谷·“华为杯”第十九届中国研究生

数学建模竞赛

题 目 方形件组批优化问题

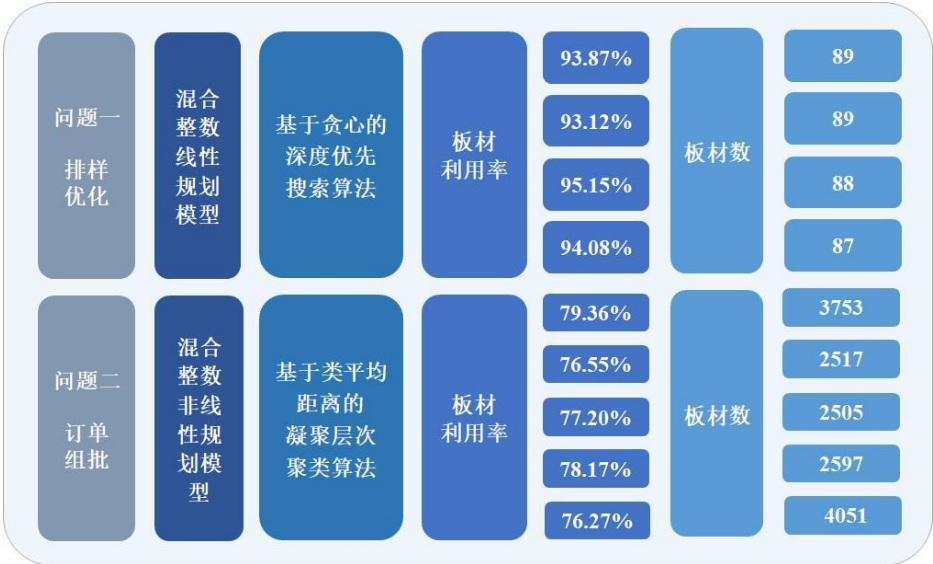
摘 要：

“中国制造 2025”将智能制造列为主攻方向，尤以个性化定制为主要竞争点。以板材为主要原片的方形件产品具有可分散加工、可灵活组装且同类产品款式极多，以及企业订单批量小、品种多、规模大的特点。如何通过订单组批与排样优化，来提高材料利用率，降低原材料消耗，从而减少资源和能源浪费，是企业生产中亟待解决的关键问题。

问题一的排样优化问题，本质是一个二维矩形切割优化问题，在满足“一刀切”三阶段精确切割的约束下，设计合理的排样方案以最大化板材利用率。本文以最小化所需板材数为目标，以订单需求和切割方式为约束条件，建立混合整数线性规划模型。利用列生成的思想，设计了基于贪心的深度优先搜索算法进行求解，优先考虑大尺寸 Item 的横向排样。结果表明，对应于输入数据集 A1、A2、A3、A4，最小所需板材数分别为 89、89、88、87，板材利用率分别为 93.87%、93.12%、95.15%、94.08%，排样方案效果图见附件 A。

问题二的订单组批问题，要求将若干订单组成不同批次进行生产，以尽可能减少所需板材数量。对此，本文以最小化需要的总板材数为目标，以每批次生产效率和产能限制为约束条件，建立混合整数非线性规划模型。本文引入订单的相似度评价指标，设计了基于类平均距离的凝聚层次聚类算法，优先将相似度高的订单聚类到同一批次，并在问题一的基础上进行求解。结果表明，对应于输入数据集 B1、B2、B3、B4、B5，最小所需板材数分别为 3753 块、2517 块、2505 块、2597 块、4051 块，板材利用率分别为 79.34%、76.55%、77.20%、78.17%、76.27%，排样方案效果图见附件 B。

关键词：排样优化 订单组批 混合整数规划 贪心法 深度优先搜索 聚类算法



目 录

1. 问题重述	3
1.1 问题背景	3
1.2 问题提出	3
1.2.1 问题一：排样优化问题.....	3
1.2.2 问题二：订单组批问题.....	3
2. 基本假设与符号说明.....	4
2.1 基本假设	4
2.2 符号说明	4
3. 问题一的模型建立与求解.....	6
3.1 问题分析	6
3.2 模型建立	8
3.2.1 决策变量.....	8
3.2.2 目标函数.....	8
3.2.3 约束条件.....	8
3.3 算法求解：基于贪心的深度优先搜索算法.....	9
3.3.1 基本定义.....	10
3.3.2 算法流程.....	10
3.3.3 算法分析.....	15
3.4 结果分析	16
4. 问题二的模型建立与求解.....	17
4.1 问题分析	17
4.2 模型建立	17
4.2.1 决策变量.....	17
4.2.2 目标函数.....	17
4.2.3 约束条件.....	17
4.3 算法求解：基于类平均距离的凝聚层次聚类算法.....	18
4.3.1 订单相似度分析.....	18
4.3.2 基本定义.....	19
4.3.3 算法流程.....	19
4.3.4 算法分析.....	20
4.4 结果分析	20
5. 模型评价与展望.....	22
5.1 创新点	22
5.2 优点	22
5.3 缺点	22
5.4 展望	22
参考文献	23
附录	24
附录 A：问题一排样方案效果图.....	24
附录 B：问题二排样方案效果图.....	28
附录 C：程序主要模块及数据结构介绍	56
附录 D：程序代码.....	57

1. 问题重述

1.1 问题背景

“中国制造 2025”将智能制造列为主攻方向，尤以个性化定制为主要竞争点。方形件产品（也称板式类产品）是以板材为主要原片、通过将平面加工后获得的几种板式配件装配而成的一类产品，具有可分散加工、可灵活组装且同类产品款式极多，以及企业订单批量小、品种多、规模大的特点，因此其生产组织多采用“订单组批+批量生产+订单分拣”的模式，这一模式的实施中订单组批与排样优化至关重要。

订单组批是将不同的订单组成若干批次，以实现订单的批量化生产。过小的组批批次降低了材料利用率，导致生产效率的降低；而过大的组批批次虽提高了材料利用率，却使得订单分拣难度提高，交货期难以保证，因此需要在个性化与生产高效性之间做权衡。

排样优化本质上是一个下料问题（也称切割填充问题），通过合理规划元件在原片上的布局，以提高原材料利用率，减少资源浪费。其中二维切割问题已被证明是 NP-Hard 问题^[1]，国内外学者已进行了大量的研究^{[2][3]}，但至今仍没有通用的标准解法^{[4][5]}。

综上，如何通过订单组批与排样优化，设计个性化定制生产模式，来提高材料利用率，降低原材料消耗，从而减少资源和能源浪费，是企业生产中亟待解决的关键问题。

1.2 问题提出

本文主要研究方形件产品的订单组批和排样优化问题，由以下两个子问题组成，其中问题二的约束基于问题一并与之相容。

1.2.1 问题一：排样优化问题

该题考虑的排样优化问题为具有“一刀切”约束的三阶段精确二维矩形切割问题，根据同一生产批次内方形件的尺寸与数量，选择原片的规格和数量，进行下料排样优化，以最大化板材原片利用率。

优化目标：最小化所需板材原片的总数。

约束约束：①同一 Stack 里的 Item 的宽度（或长度）相同；

②不允许通过拼接得到最终的 Item，必须是完整的。

1.2.2 问题二：订单组批问题

该题主要考虑的是方形件产品的订单组批问题，即在考虑订单交货期、设备产能负荷、仓储容量、材料利用率、生产效率、生产工艺约束等因素下，对生产订单进行组批优化；将具有相同材质、交货期相近、工艺相似的订单安排在同一个生产批次，以保证交货期，尽可能提高原材料的利用率，提高设备生产效率。

优化目标：最小化所需板材原片的总数。

约束条件：在问题一基础上，满足：

① 每份订单当且仅出现在一个批次中；

② 每个批次中相同材质的 Item 才能使用同一块板材进行排样；

③ 每个批次的 Item 总数不超过 1000；

④ 每个批次的 Item 面积总和不超过 $250m^2$ 。

2. 基本假设与符号说明

2.1 基本假设

- (1) 本题只考虑“一刀切”的切割方式，即直线切割，切割方向必须垂直于板材的一条边，且每次切割都将板材分离成两块；
- (2) 切割阶段数小于等于 3，且同一阶段的切割方向相同；
- (3) 每阶段的切割刀数没有限制；
- (4) 排样方式为精确排样；
- (5) 所有材质的板材原片仅有一种规格且数量充足；
- (6) 排样方案不考虑锯缝宽度影响，即切割的缝隙宽度可忽略不计；
- (7) 所有订单的交货期一致。

2.2 符号说明

表 1 符号及其含义

符号	含义
Plate	原片
Stripe	第 1 阶段切割生成的模块（即条带）
Stack	第 2 阶段切割生成的模块（即栈）
Item	第 3 阶段切割生成的模块（即产品项）
$J(A, B)$	杰卡德相似系数
$J_\delta(A, B)$	杰卡德距离
$\lceil \cdot \rceil$	向上取整
$ A $	集合 A 的元素个数

表 2 索引及其含义

索引	含义
i, n	Item 序号
j	批次数
k	订单序号
p	材质序号

表 3 决策变量及其含义

决策变量	含义
y_b	$y_b = 1$ 表示原片 b 被使用，否则 $y_b = 0$
u_{sb}	$u_{sb} = 1$ 表示 Plate b 的 Stripe s 被使用，否则 $u_{sb} = 0$
l_{sb}	Plate b 的 Stripe s 的长度
w_{msb}	Plate b 的 Stripe s 的 Stack m 的宽度
x_{imsb}	$x_{imsb} = 1$ 表示 Item i 属于 Plate b 的 Stripe s 的 Stack m ，否则 $x_{imsb} = 0$
x_{imsb}^r	$x_{imsb}^r = 1$ 表示旋转 90° 后的 Item i 属于 Plate b 的 Stripe s 的 Stack m ，否则 $x_{imsb}^r = 0$
Y_{pj}	批次 j 中材质为 p 的所有 Item 排样所需的最小板材数量
a_{kj}	$a_{kj} = 1$ 表示订单 k 属于批次 j ，否则 $a_{kj} = 0$
G_{pj}	批次 j 中材质为 p 的 Item 的尺寸的集合

表 4 参数及其含义

参数	含义
L	原片长度
W	原片宽度
l_i	Item i 的长度
w_i	Item i 的宽度
B	可用原片的最大数量
S	一个原片切割得到 Stripe 的最大数量
M	一个 Stripe 切割得到 Stack 的最大数量
N	Item 的总数
Y_0	理论最小所需板材数量
η_0	板材极限利用率
η	板材利用率
N_{max}	单个批次 Item 总数上限
A_{max}	单个批次 Item 面积总和上限
Y_{pj}	批次 j 中材质为 p 的所有 Item 排样所需的最小板材数量
K	总订单数
P_k	订单 k 中的材质总数
N_{pk}	订单 k 中材质为 p 的 Item 总数
l_{npk}	订单 k 中材质为 p 的 Item n 的长度
w_{npk}	订单 k 中材质为 p 的 Item n 的宽度

3. 问题一的模型建立与求解

3.1 问题分析

对于问题一的排样优化问题，其本质是一个二维矩形切割问题，要求在满足相关约束的条件下，合理规划 Item 在板材原片上的分布，以最小化板材原片的用量，提高板材原片的利用率。

国内外学者针对排样问题已经展开了大量的研究，1979 年 Garey 等人用数学方法严格证明了二维切割问题是 NP 完全问题^[1]，一些精确方法仅适用于中小型问题的求解^{[6][7]}。对于大规模二维切割问题，启发式算法能在合理时间内获得高质量的解，因而得到了广泛的应用^{[8][9]}。常用的启发式算法包括模拟退火算法、遗传算法、粒子群算法、基于树的搜索算法等。总体而言，排样问题由于其复杂度和广泛性，当前研究呈现较强的领域依赖性，因此应该有针对性地研发各领域专用排样算法，而无需研究通用的算法^[10]。

问题一中要求满足“一刀切”约束，且限制阶段数不能超过 3，这就要求我们用连续平行的层组成布局方案。考虑一个“自底向上”的思想，大致思路是先将长度或者宽度相等的 Item 进行拼接，形成一个较大的 Stack；然后再将这些 Stack 进行组合，并添加适当冗余，形成 Stripe；最后将 Stripe 进行组合，并添加适当冗余，形成原片。

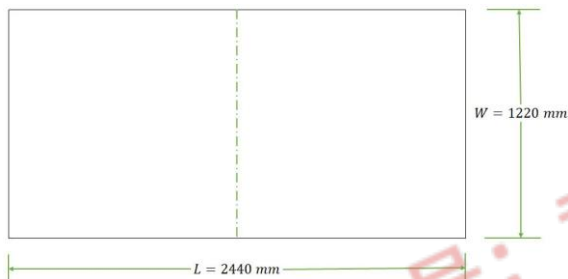


图 1 横向摆放的原片

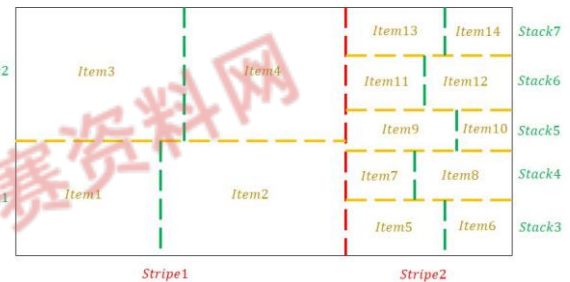
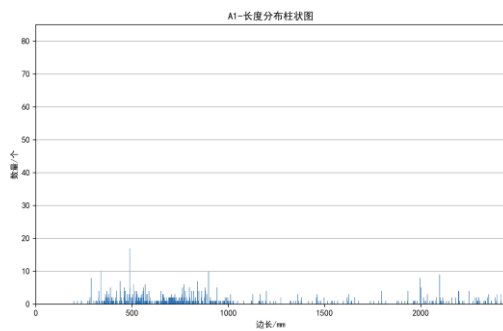


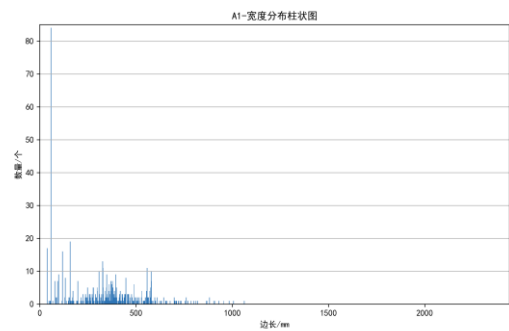
图 2 三阶段切割示意图

为方便描述，我们规定所有模块放置时，横向为长度，纵向为宽度，初始状态调整板材原片和 Item 方向为横向摆放（即长度大于宽度）；其中原料板材方向固定不变（如图 1），Item 可旋转 90° 。如图 2，在对原片进行切割时，我们规定第 1 阶段纵向切割（红色虚线），第 2 阶段横向切割（绿色虚线），第 3 阶段纵向切割（黄色虚线），这样极大地简化了问题。在此规定下，所有 Stripe 的纵向长度等于原片的宽度 W ，所有 Stack 的长度等于所属 Stripe 的长度。

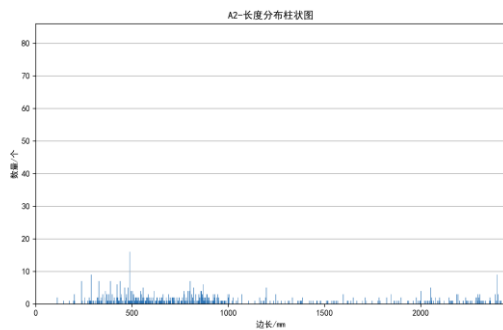
观察图 3 中数据集中 Item 的长宽分布情况可以发现，数据中宽度相等的 Item 明显多于长度相等的情况。这使得在对相同边长的 Item 进行拼接时，会更多地宽度相同的 Item 横向拼接，形成长条形的 Stack（即长度大于宽度）。因此，我们在将 Stack 放置到 Stripe 中时，基于贪心的思想，优先考虑将长度最大的 Stack 横向放入，然后再放置次长的 Stack，以此类推，直到该 Stripe 再也放不下任何 Stack 为止，选择其中最大的 Stack 的宽度作为该 Stripe 的宽度。直观上这样的操作能获得更高的板材利用率，因此上述规定的“纵向-横向-纵向”3 阶段切割顺序具有一定的合理性，能够在极大降低问题复杂度的同时，保证最优切割方案的可靠性。



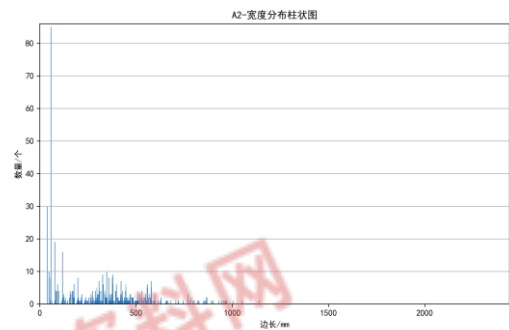
(1) dataA1 中 Item 长度分布情况



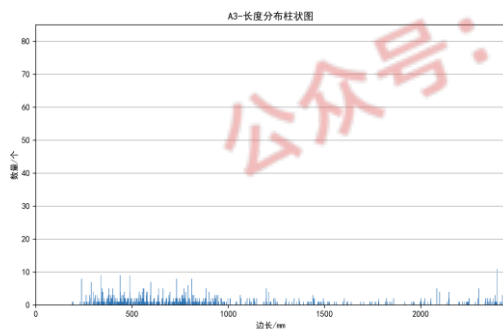
(2) dataA1 中 Item 宽度分布情况



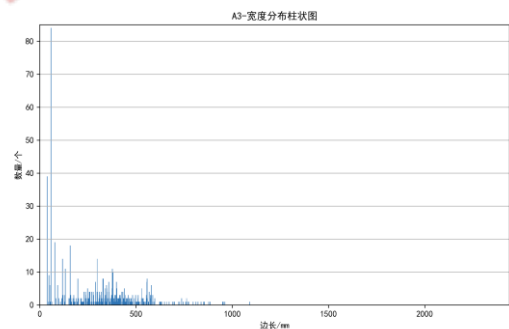
(3) dataA2 中 Item 长度分布情况



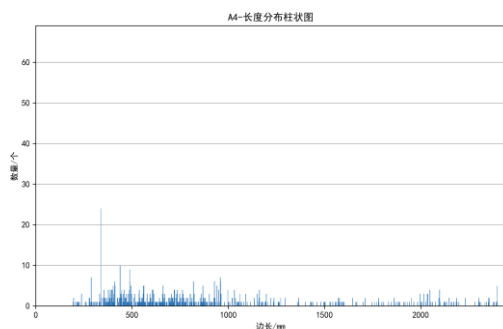
(4) dataA2 中 Item 宽度分布情况



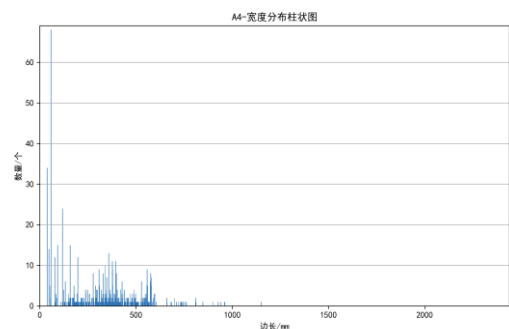
(5) dataA3 中 Item 长度分布情况



(6) dataA3 中 Item 宽度分布情况



(7) dataA4 中 Item 长度分布情况



(8) dataA4 中 Item 宽度分布情况

图 3 各数据集中 Item 的长宽分布情况

本题中假设原片的数量没有上限，因此可以将原片的最大数量 B 设为 Item 的总数 N ，即每个原片切割只得到一个 Item 的最差情况。本题的优化目标为最小化板材原料用量，需要满足的约束条件如下：

- ① 满足订单需求，即订单中所有的 Item 都必须被完整切割出来；
- ② 所有 Item 的宽度（或旋转 90° 后的 Item 的长度）与所属 Stack 的宽度相等；
- ③ 同一 Stack 的所有 Item 的长度（或旋转 90° 后的 Item 的宽度）之和不超过所属 Stripe 的长度；
- ④ 同一 Stripe 的所有 Stack 的宽度之和不超过原片的宽度 W ；
- ⑤ 同一原片的 Stripe 的长度之和小于原片的长度 L 。

根据以上描述，我们基于[11]的三阶段二维矩形切割模型，建立了一个与原问题等价的**混合整数线性规划模型**。该模型中虽然约束数目有限，但变量的规模随着数据规模的增大呈爆炸式增长。所以对于这样一个 NP-Complete 问题，难以寻求精确的求解方法，设计能够在有效时间内获得高质量解的启发式算法才是可行之道。因此，本文运用列生成的思想，提出了一个**基于贪心的深度优先搜索算法**。这一算法从根本上排除了原问题中一些不合理的切割方案，缩减了模型中的变量数量，从而显著降低了计算规模。

3.2 模型建立

3.2.1 决策变量

l_{sb} : Plate b 的 Stripe s 的长度

w_{msb} : Plate b 的 Stripe s 的 Stack m 的宽度

$$y_b = \begin{cases} 1, & \text{Plate } b \text{ 被使用} \\ 0, & \text{Plate } b \text{ 未被使用} \end{cases}, b = 1, 2, \dots, B$$

$$u_{sb} = \begin{cases} 1, & \text{Plate } b \text{ 的 Stripe } s \text{ 被使用} \\ 0, & \text{Plate } b \text{ 的 Stripe } s \text{ 未被使用} \end{cases}, s = 1, 2, \dots, S, b = 1, 2, \dots, B$$

$$x_{imsb} = \begin{cases} 1, & \text{Item } i \text{ 属于 Plate } b \text{ 的 Stripe } s \text{ 的 Stack } m \\ 0, & \text{Item } i \text{ 不属于 Plate } b \text{ 的 Stripe } s \text{ 的 Stack } m \end{cases}$$

$i = 1, 2, \dots, N, m = 1, 2, \dots, M, s = 1, 2, \dots, S, b = 1, 2, \dots, B$

$$x_{imsb}^r = \begin{cases} 1, & \text{旋转 } 90^\circ \text{ 后的 Item } i \text{ 属于 Plate } b \text{ 的 Stripe } s \text{ 的 Stack } m \\ 0, & \text{旋转 } 90^\circ \text{ 后的 Item } i \text{ 不属于 Plate } b \text{ 的 Stripe } s \text{ 的 Stack } m \end{cases}$$

$i = 1, 2, \dots, N, m = 1, 2, \dots, M, s = 1, 2, \dots, S, b = 1, 2, \dots, B$

3.2.2 目标函数

以最小化使用的原片数量为目标，问题一的目标函数如下：

$$\text{Min} \sum_{b=1}^B y_b \quad (1)$$

3.2.3 约束条件

a) 所有的 Item 都必须从原料中被切割出来：

$$\sum_{b=1}^B \sum_{s=1}^S \sum_{m=1}^M (x_{imsb} + x_{imsb}^r) = 1, \forall i \quad (2)$$

b) 所有 Item 的宽度（或旋转 90° 后的 Item 的长度）与所属的 Stack 的宽度相等：

$$w_i x_{imsb} + l_i x_{imsb}^r = w_{msb}, \forall i, \forall m, \forall s, \forall b \quad (3)$$

- c) 同一 Stack 的所有 Item 的长度（或旋转 90° 后的 Item 的宽度）之和不超过所属 Stripe 的长度：

$$\sum_{i=1}^N (l_i x_{imsb} + w_i x_{imsb}^r) \leq l_{sb}, \forall m, \forall s, \forall b \quad (4)$$

- d) 同一 Stripe 的所有 Stack 的宽度之和不超过原片的宽度 W ：

$$\sum_{m=1}^M w_{msb} \leq W u_{sb}, \forall s, \forall b \quad (5)$$

- e) 同一原片的 Stripe 的长度之和小于原片的长度 L ：

$$\sum_{s=1}^S l_{sb} \leq L y_b, \forall b \quad (6)$$

- f) 对所有被使用的原片进行编号：

$$y_b \geq y_{b+1}, \forall b \quad (7)$$

综上，针对问题一建立如下的混合整数线性规划模型：

$$\begin{aligned} & \text{Min} \sum_{b=1}^B y_b \\ & \text{s.t.} \left\{ \begin{aligned} & \sum_{b=1}^B \sum_{s=1}^S \sum_{m=1}^M (x_{imsb} + x_{imsb}^r) = 1, \forall i \\ & w_i x_{imsb} + l_i x_{imsb}^r = w_{msb}, \forall i, \forall m, \forall s, \forall b \\ & \sum_{i=1}^N (l_i x_{imsb} + w_i x_{imsb}^r) \leq l_{sb}, \forall m, \forall s, \forall b \\ & \sum_{m=1}^M w_{msb} \leq W u_{sb}, \forall s, \forall b \\ & \sum_{s=1}^S l_{sb} \leq L y_b, \forall b \\ & y_b \geq y_{b+1}, \forall b \\ & y_b, u_{sb}, x_{imsb}, x_{imsb}^r \in \{0, 1\}, \forall i, \forall m, \forall s, \forall b \end{aligned} \right. \end{aligned}$$

3.3 算法求解：基于贪心的深度优先搜索算法

排样优化问题属于典型的 NP-Hard 问题，无法在多项式时间内求得最优解。3.2.3 小节提出的混合整数线性优化问题，其决策变量既有离散的整数变量，又有连续变量。虽然该问题的目标函数和约束都是线性的，且约束个数也有限，但由于其含有 4 层索引嵌套的变量（如 x_{imsb} 和 x_{imsb}^r ），这将导致变量数量会随着问题规模的扩大呈现爆炸式增长，常用的商业求解器如 Gurobi、CPLEX 根本无法负载。相比之下，启发式算法在解决 NP-Hard 问题中具有很强的适应性，能够在有效时间内获得高质量的解。本文针对问题一，结合列生

成的思想，提出了基于贪心的深度优先搜索算法。

3.3.1 基本定义

表 5 符号及其含义

符号	含义
IS	所有 Item 组成的集合
ISN	IS 中所有还未填充的 Item 形成的集合
ISY	IS 中所有已经填充的 Item 形成的集合
$Plate_i^m$	第 <i>i</i> 张板材原片在第 <i>m</i> 轮填充时的原材料
XL_i^m	原材料 $Plate_i^m$ 的横向长度
YL_i^m	原材料 $Plate_i^m$ 的纵向长度
$Item_j$	ISN 中第 <i>j</i> 个 Item
l_j	$Item_j$ 的较长边的长度大小
w_j	$Item_j$ 的较短边的长度大小

根据表 5 中的定义，显然有 $IS = ISY \cup ISN$ 。原材料 $Plate_i^m$ 摆放方向始终满足纵向长度为 $YL_i^m = W = 1220\text{mm}$ 。 $m = 1$ 时，横向长度 $XL_i^1 = L = 2440\text{mm}$ 。

本题所提算法应用“左下策略”在原材料中摆放 Item。设当前 $Plate_i^m$ 的纵向剩余长度为 RYL_i^m ，纵向已用长度 UYL_i^m ；横向剩余长度为 RXL_i^m ，横向已用长度为 UXL_i^m 。

$$RYL_i^m + UYL_i^m = YL_i^m$$

$$RXL_i^m + UXL_i^m = XL_i^m$$

第*m*轮填充还未开始时， $RYL_i^m = YL_i^m = W$ ， $UYL_i^m = 0$ ； $RXL_i^m = XL_i^m$ ， $UXL_i^m = 0$ 。

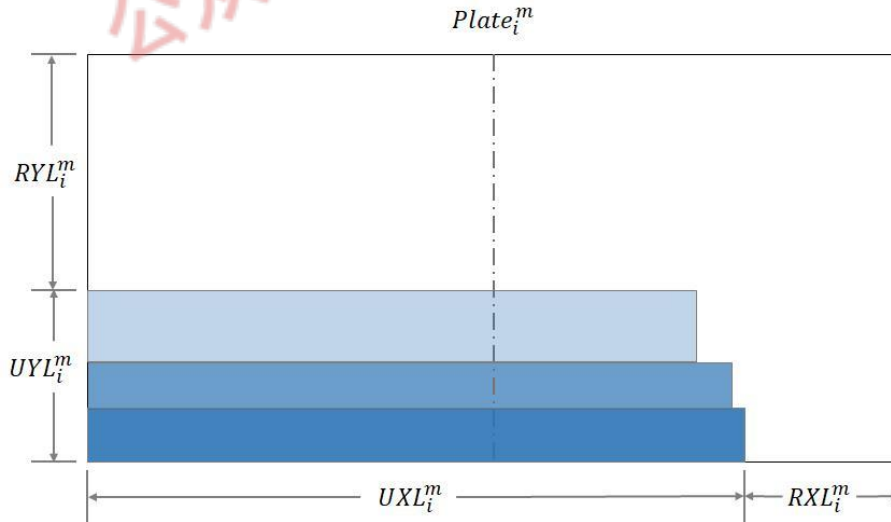


图 4 第*i*张原片在第*m*轮填充时的原材料 $Plate_i^m$ 示意图

3.3.2 算法流程

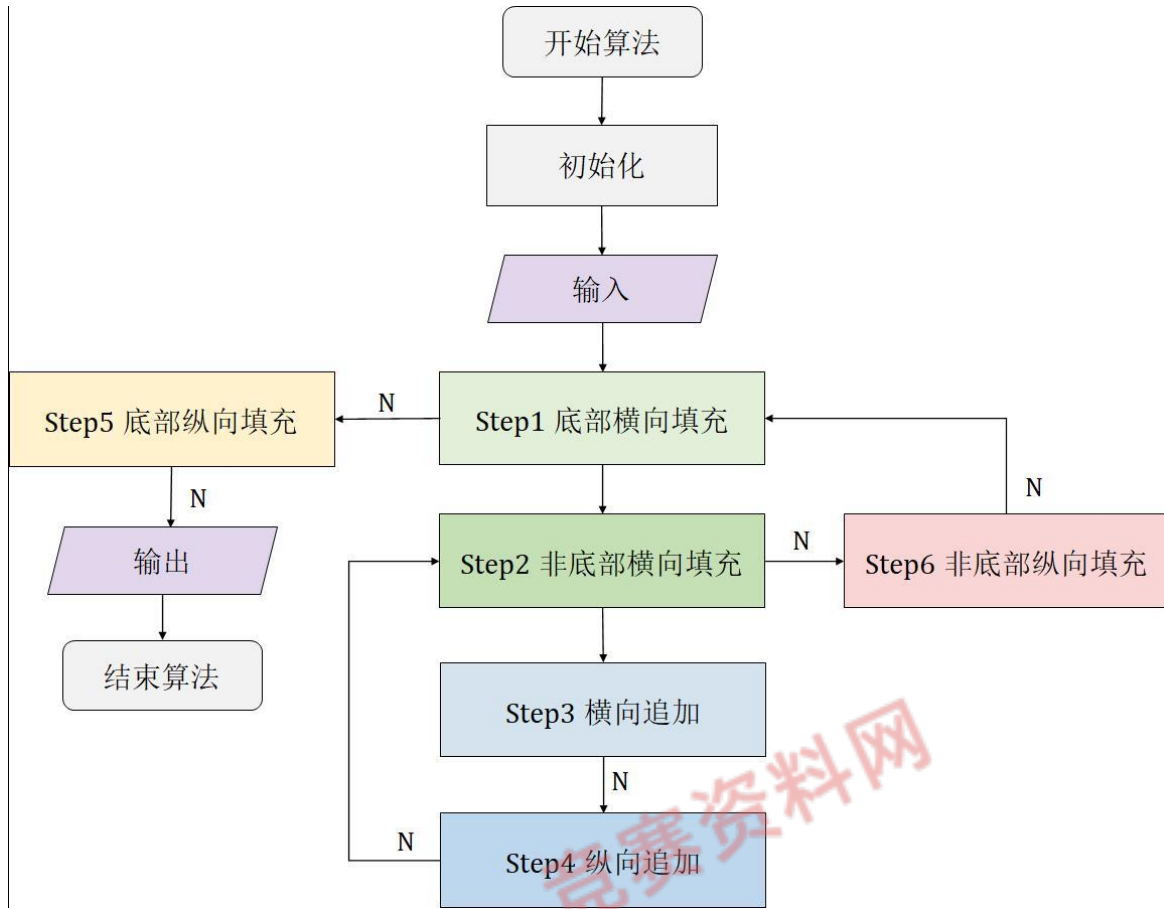


图 5 基于贪心的深度优先搜索算法框架图

图 5 展示了基于贪心的深度优先搜索算法框架图，具体算法流程如下：

#Step0 【初始化】

将 IS 中的 Item 依据长度降序排列；若存在长度相同的多个 Item，则在其内部依据宽度降序排列。

初始状态下， $ISN = IS$ ， $ISY = \emptyset$ 。令 $i = 1$ ， $m = 1$ ，开始第 1 块板材原片 $Plate_i^m$ 的第 1 轮填充。

#Step1 【底部横向填充】

搜索 $j^* = \arg \max_{j \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} l_j$ ，即在所有未填充的 $Item_j (j = 1, \dots, |ISN|)$ 中搜索满足条件 $l_j \leq XL_i^m$ ， $w_j \leq YL_i^m$

件 $\begin{cases} l_j \leq XL_i^m \\ w_j \leq YL_i^m \end{cases}$ 、长度最长的 $Item_{j^*}$ 。

如果 $Item_{j^*}$ 存在，则 $Item_{j^*}$ 完成横向填充，更新集合 ISY 、 ISN ，更新横长 RXL_i^m 、 UXL_i^m ，纵长 RYL_i^m 、 UYL_i^m 。执行 Step2。

如果 $Item_{j^*}$ 不存在，执行 Step6。

#Step2 【非底部横向填充】

搜索 $j^* = \arg \max_{j \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} l_j$, 即在所有未填充的 $Item_j (j = 1, \dots, |ISN|)$ 中搜索满足条件 $l_j \leq UXL_i^m, w_j \leq RYL_i^m$

件 $\begin{cases} l_j \leq UXL_i^m \\ w_j \leq RYL_i^m \end{cases}$ 、长度最长的 $Item_{j^*}$ 。

如果 $Item_{j^*}$ 存在, 则 $Item_{j^*}$ 完成横向填充, 更新集合 ISY 、 ISN , 更新纵长 RYL_i^m 、 UYL_i^m 。

令 $bl = l_{j^*}$, 执行 Step3。

如果 $Item_{j^*}$ 不存在, 则执行 Step5。

#Step3 【横向追加】

搜索 $k^* = \arg \max_{k \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} l_k$, 即在所有未填充的 $Item_k (k = 1, \dots, |ISN|)$ 中搜索满足条件 $w_k = w_j, bl + l_k \leq UXL_i^m$

件 $\begin{cases} w_k = w_j \\ bl + l_k \leq UXL_i^m \end{cases}$ 、长度最长的 $Item_{k^*}$ 。

如果 $Item_{k^*}$ 存在, 则 $Item_{k^*}$ 完成横向填充, 更新集合 ISY 、 ISN 。令 $bl = bl + l_{k^*}$, 重复 Step3。

如果 $Item_{k^*}$ 不存在, 执行 Step4。

#Step4 【纵向追加】

搜索 $k^{**} = \arg \max_{k \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} w_k$, 即在所有未填充的 $Item_k (k = 1, \dots, |ISN|)$ 中搜索同时满足条件 $l_k = w_j, bl + w_k \leq UXL_i^m$

满足条件 $\begin{cases} l_k = w_j \\ bl + w_k \leq UXL_i^m \end{cases}$ 、宽度最长的 $Item_{k^{**}}$ 。

如果 $Item_{k^{**}}$ 存在, 则 $Item_{k^{**}}$ 完成纵向填充, 更新集合 ISY 、 ISN 。令 $bl = bl + w_{k^{**}}$, 重复 Step4。

如果 $Item_{k^{**}}$ 不存在, 更新纵长 RYL_i^m 、 UYL_i^m , 执行 Step2。

#Step5 【非底部纵向填充】

搜索 $h^* = \arg \max_{h \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} w_h$, 即在所有未填充的 $Item_h (h = 1, \dots, |ISN|)$ 中搜索满足条件 $l_h = RYL_i^m, w_h \leq UXL_i^m$

条件 $\begin{cases} l_h = RYL_i^m \\ w_h \leq UXL_i^m \end{cases}$ 、宽度最长的 $Item_{h^*}$ 。

如果 $Item_{h^*}$ 存在, 则 $Item_{h^*}$ 完成纵向填充, 更新集合 ISY 、 ISN 。令 $bl = bl + w_{h^*}$, 重复 Step5。

如果 $Item_{h^*}$ 不存在, 更新纵长 RYL_i^m 、 UYL_i^m 。 $Plate_i^m$ 完成第 m 轮填充, 将右侧余料剪裁出来。此时有:

① 若 $m > 1$ 或 $i > V$, 则令 $m = m + 1$, 直接将右侧余料作为下一轮的原材料 $Plate_i^m$,

执行 Step1（如图 6）；

- ② 若 $m = 1$, $\max_{j \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} l_j \geq W$, 则增加一块板材原片, 即 $i = i + 1$, 执行 Step1（如图 7）；
- ③ 若 $m = 1$, $\max_{j \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} l_j < W$, 且原材料 $Plate_i^1$ 中存在 $l_j \geq W$ 的 $Item_j$, 则暂时不再新增板材原片, 记录已经使用的板材原片数量 V 。令 $m = m + 1$, 收集第 1 轮的所有余料 $Plate_i^2 (i = 1, \dots, V)$, 并根据其横向长度 XL_i^2 进行降序排列。令 $i = 1$, 执行 Step1。

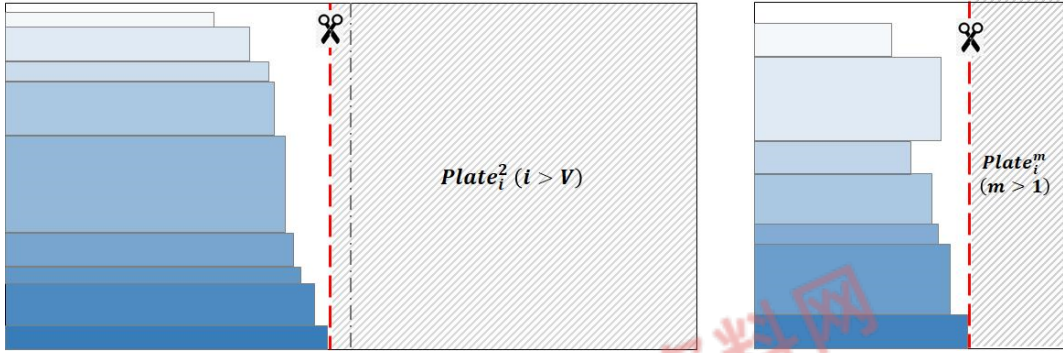


图 6 Step5 状态①示意图

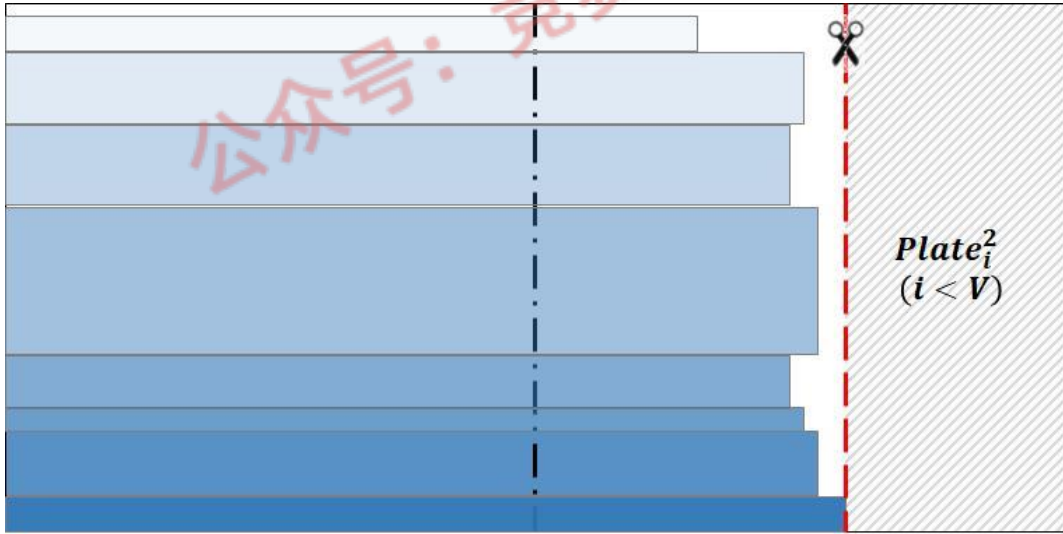


图 7 Step5 状态②示意图

#Step6 【底部纵向填充】

搜索 $h^{**} = \arg \max_{h \in \{1, \dots, |ISN|\} \text{ if } ISN \neq \emptyset} l_j$, 即在所有未填充的 $Item_h (h = 1, \dots, |ISN|)$ 中搜索满足 $w_h \leq XL_i^m, l_h \leq W$

条件 $w_h \leq XL_i^m$ 、长度最长的 $Item_{h^{**}}$ 。

如果 $Item_{h^{**}}$ 存在, 则 $Item_{h^{**}}$ 完成纵向填充, 更新 ISY 、 ISN ; 更新横长 RXL_i^m 、 UXL_i^m , 纵长 $RYL_i^m = W$ 、 UYL_i^m ; 重复 Step6。

如果 $Item_{h^{**}}$ 不存在, 则第 i 块 plate 完成全部填充。若此时 $ISN = \emptyset$, 结束算法, 输出板材切割结果并计算板材利用率。若此时 $ISN \neq \emptyset$, 则有:

- ① 若 $i < V$ ，则令 $i = i + 1$ ，以余料 $Plate_i^m (m > 1)$ 作为原材料，执行 Step1；
- ② 若 $i \geq V$ ，则令 $i = i + 1$ ，再取一块新的板材原片 $Plate_i^1$ 作为原材料，执行 Step1。

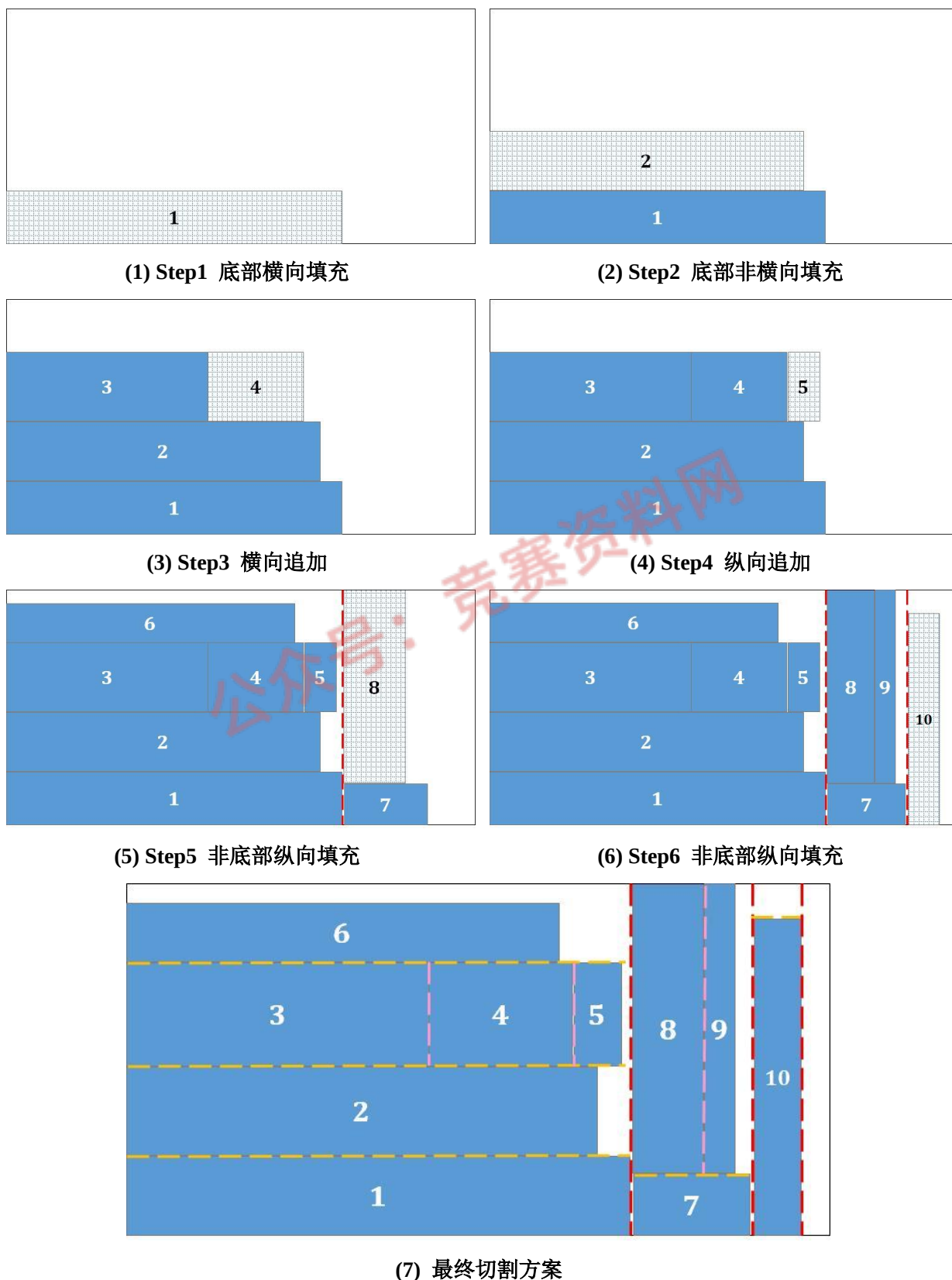


图 8 基于贪心的深度优先搜索算法应用举例

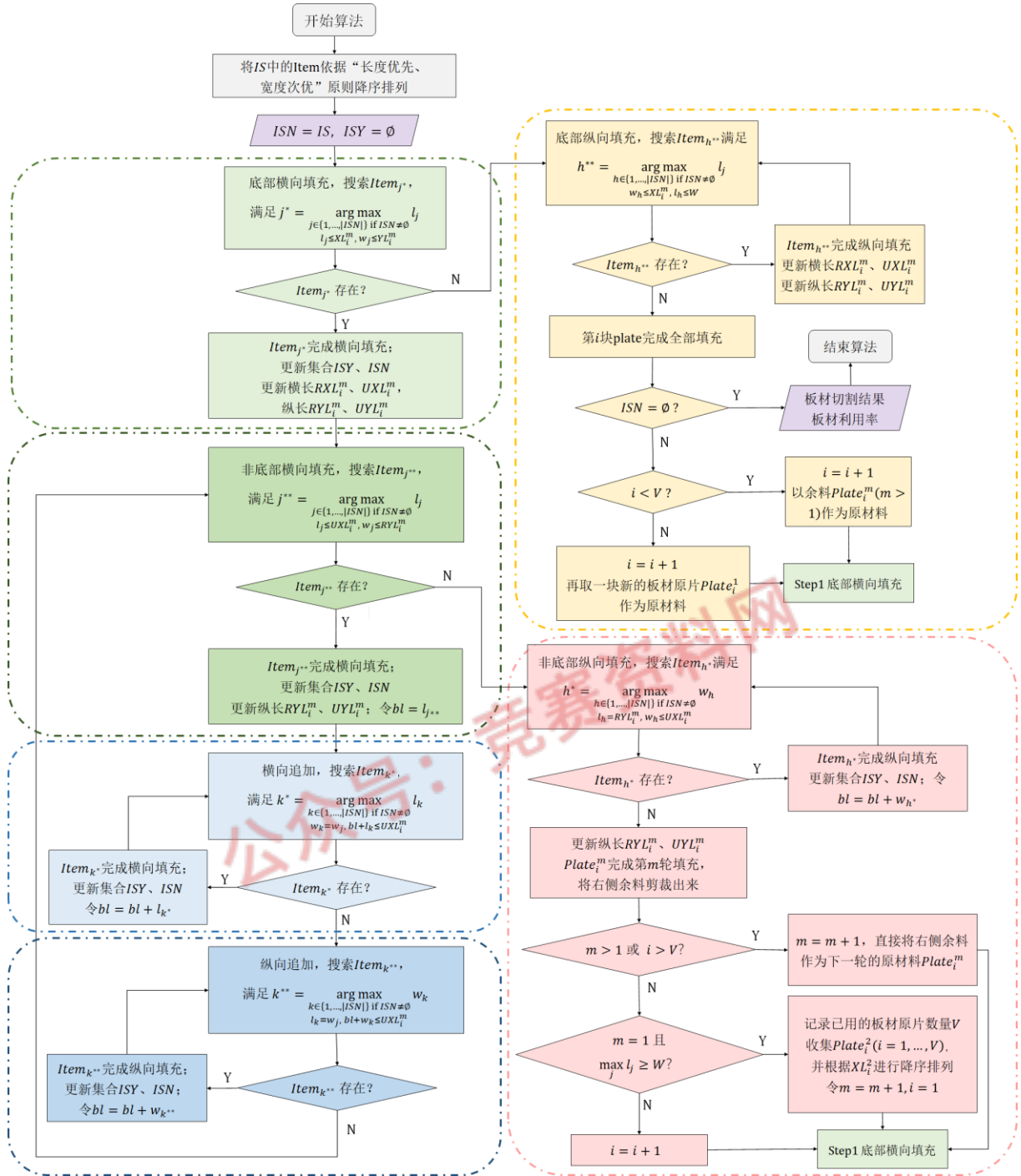


图 9 基于贪心的深度优先搜索算法流程图

3.3.3 算法分析

该算法主要的填充策略：①优先横向填充 Item；②优先填充大尺寸 Item。策略①基于数据文件中的 Item 尺寸，即宽度相等的 Item 明显多余长度相等的 Item。策略②基于贪心思想，因为某些大尺寸 Item 一定需要完整的板材原片来切割，优先处理大尺寸 Item 可以使得某些小尺寸 Item 填充在其余料中，从而充分利用原材料。

通过分析，上述算法的计算复杂度为 $o(n^2)$ ，其中 n 表示 Item 的总数，其嵌套主要体现在 Item 的填充与追加过程。具体而言，Item 每次在完成填充后，会遍历所有未填充 Item

来搜索是否有合适的 Item 能够追加在其右侧。

该算法的 PC 计算平台配置为: Inter(R) Core(TM) i7-8750H Ryzen 芯片的 CPU, 2.20GHz 主频, 16GB 内存, Windows11 21H2 64 位家庭版的操作系统。针对问题一的输入数据集 A1、A2、A3、A4, 算法分别在用时 **6.20s**、**5.26s**、**8.05 s**、**7.12 s** 后完成切割任务, 并输出排样方案效果图和板材利用率。由此可见, 针对二维排样这一 NP-Hard 问题, 该算法虽然是实用性的启发式算法, 但足够简便灵巧, 能够保证在较短时间内求解出接受度高的结果。

3.4 结果分析

首先计算理论最小所需板材数量 Y_0 。在数据集 A 中, 计算所有 Item 的面积总和, 直接除以原片的面积, 并向上取整。计算公式如下:

$$Y_0 = \left\lceil \left(\sum_{i=1}^N l_i w_i \right) / (LW) \right\rceil \tag{8}$$

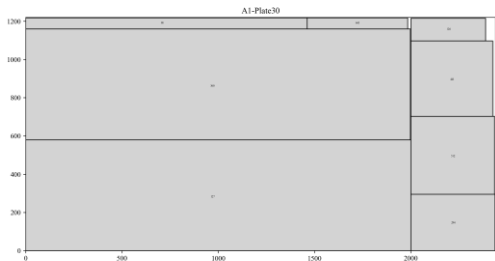
则板材极限利用率 η_0 计算公式如下:

$$\eta_0 = \left(\sum_{i=1}^N l_i w_i \right) / (LW Y_0) \tag{9}$$

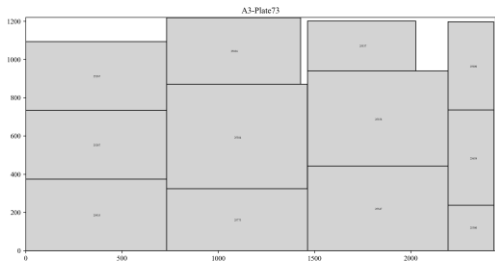
通过上述提出的算法, 输入 A 组数据, 计算得到问题一的优化结果如表 6 所示。其中对应于输入数据集 A1、A2、A3 和 A4, 所需的最少板材数 Y 分别为 **89 块**、**89 块**、**88 块**和 **87 块**, 板材利用率 η 分别为 **93.87%**、**93.12%**、**95.15%**和 **94.08%**, 图 10 展示了其中的部分排样方案效果图, 完整的排样方案效果图见附录 A。

表 6 问题一优化结果

数据集 序号	理论最小所需 板材数 Y_0 /块	板材极限利用率 η_0	最小所需 板材数 Y /块	板材利用率 η
A1	84	98.28%	89	93.87%
A2	83	98.66%	89	93.12%
A3	84	98.50%	88	95.15%
A4	82	98.62%	87	94.08%



(1) A1-Plate 30



(2) A3-Plate 73

图 10 问题一排样方案效果图 (部分)

4. 问题二的模型建立与求解

4.1 问题分析

问题二是一个订单组批问题，要求对数据集中的订单进行组批，再对每个批次分别进行排样优化，在满足订单需求和相关约束条件下，最小化所需板材原料的总数。

该题中，所有订单的交货期一致，因此不需要考虑订单完成的先后顺序，只需满足一份订单当且仅当出现在一个批次中。每一批次的原料仅供本批次使用，余料不能用于下一批次生产。通常而言，在某一批次生产过程中，同种原料的最后一块板材原件浪费最多。如果能够将材质相同的产品放在同一批次生产，至少可以显著降低最后一块板材原件的浪费。因此，问题二的组批过程中应该尽可能地将产品材质相近的订单组为同一批次。

为方便描述，我们将 3.2.3 小节中问题一的数学模型写成以下函数：

$$Y_{pj} = F(G_{pj}) \quad (10)$$

其中函数的自变量 G_{pj} 表示批次 j 中材质为 p 的 Item 的尺寸的集合，因变量 Y_{pj} 表示根据问题一排样优化后得到的最小所需板材总数，即批次 j 中材质为 p 的 Item 最小所需板材总数。

针对问题二，以最小化所有批次所有材质所需的原片总数为优化目标，以每批次生产效率和产能限制为约束条件，建立**混合整数非线性规划模型**。该模型对应的订单组批问题同样是一个 NP-Hard 问题。

以问题二的数据集 B1 为例，其中有来源于 546 个订单、涉及 130 种原材料的 26811 件产品，数据规模大且结构复杂。而聚类分析广泛应用于挖掘大规模数据之间的隐藏联系，其基于“物以类聚”的道理，将一组数据集按照某些相似度函数或相似性规则划分为不同类别，这些类别被称作“簇”。据此，本文以订单中产品材质的相似度为指标，应用**基于类平均距离的凝聚层次聚类算法**对订单进行组批。

4.2 模型建立

4.2.1 决策变量

$$a_{kj} = \begin{cases} 1, & \text{订单} k \text{ 属于批次 } j \\ 0, & \text{订单} k \text{ 不属于批次 } j \end{cases}$$

4.2.2 目标函数

以最小化使用的原片数量为目标，即最小化所有批次中所有材质所需的板材数量总和，问题二的目标函数如下：

$$\text{Min} \sum_{j=1}^J \sum_{p=1}^{P_k} Y_{pj} \quad (11)$$

4.2.3 约束条件

a) 所有的订单当且仅当出现在一个批次中：

$$\sum_{j=1}^J a_{kj} = 1, \quad \forall k \quad (12)$$

b) 一个批次中所有的 Item 数量之和不超过上限 N_{max} ：

$$\sum_{k=1}^K \sum_{p=1}^{P_k} a_{kj} N_{pk} \leq N_{max}, \quad \forall j \quad (13)$$

c) 一个批次中所有的 Item 面积之和不超过上限 A_{max} ：

$$\sum_{k=1}^K \sum_{p=1}^{P_k} \sum_{n=1}^{N_{pk}} a_{kj} l_{npk} w_{npk} \leq A_{max}, \quad \forall j \quad (14)$$

综上，针对问题二建立如下的混合整数规划模型：

$$\begin{aligned} & \text{Min} \sum_{j=1}^J \sum_{p=1}^{P_k} Y_{pj} \\ & \text{s. t.} \left\{ \begin{array}{l} \sum_{j=1}^J a_{kj} = 1, \quad \forall i \\ \sum_{k=1}^K \sum_{p=1}^{P_k} a_{kj} N_{pk} \leq N_{max}, \quad \forall j \\ \sum_{k=1}^K \sum_{p=1}^{P_k} \sum_{n=1}^{N_{pk}} a_{kj} l_{npk} w_{npk} \leq A_{max}, \quad \forall j \\ Y_{pj} = F(G_{pj}) \\ a_{kj} \in \{0, 1\}, \quad \forall k, \quad \forall j \end{array} \right. \end{aligned}$$

4.3 算法求解：基于类平均距离的凝聚层次聚类算法

4.2.3 小节建立的模型是一个混合整数非线性规划模型，其中嵌套了问题一的模型，属于 NP-Hard 问题，无法在多项式有效时间内求得精确解。针对问题二，我们设计了基于类平均距离的凝聚层次聚类算法，通过对订单间的相似度进行评估，优先将相似度高的订单安排在同一批次进行生产，以此来提高板材利用率。

该算法的思想是：首先把所有的订单都作为一个类，然后将满足约束条件且相似度最高的两个类合并形成一个新的类，重复该过程，直到所有类都无法再合并为止，即完成聚类过程。

4.3.1 订单相似度分析

由于只有同一批次中相同材质的 Item 才能使用同一块板材原片进行排样，我们以订单之间的材质相似度作为评估订单相似度的主要指标，基于[12]，引入以下参数：

- a) **杰卡德相似系数**：两个订单集合A和B材质种类的交集在其并集中所占的比例，称为这两个订单集合之间的杰卡德相似系数，用符号 $J(A, B)$ 表示

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (15)$$

- b) **杰卡德距离**：两个订单集合中不同种类材质集合在其并集中所占的比例，称为这两个订单集合之间的杰卡德距离，用符号 $J_\delta(A, B)$ 表示

$$J_\delta(A, B) = 1 - J(A, B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} \quad (16)$$

两个订单集合之间的杰卡德相似系数越大（也即杰卡德距离越小），表明这两个订单集合的相似程度越高。

- c) **类平均距离**：两个类H和K之间各订单两两之间杰卡德距离的平均值，用符号 D_{HK} 表示。

4.3.2 基本定义

表 7 符号及其含义

符号	含义
T	类的总数
K	订单总数
$Q_1(n), Q_2(n), \dots, Q_T(n)$	第 n 次的聚类情况
$Plate_i^m$	第 i 张板材原片在第 m 轮填充时的原材料

4.3.3 算法流程

根据前面的分析，本文提出的基于类平均距离的凝聚层次聚类算法流程如下：

Algorithm 基于类平均距离的凝聚层次聚类算法	
1	输入：订单 $Order_k(k = 1, \dots, K)$
2	初始化：
3	分类 $Q_1(0), Q_2(0), \dots, Q_T(0)$
4	$T = K$
5	$n = 0$
6	$m = 1$
7	迭代过程：
8	while $m \leq \frac{1}{2}T(T-1)$ do
9	计算类平均距离矩阵 $D(n)$
10	找到 $D(n)$ 中的第 m 小的元素及其对应的类 $Q_x(n)$ 、 $Q_y(n)$
11	if $Q_x(n), Q_y(n)$ 满足合并约束
12	合并 $Q_x(n), Q_y(n)$
13	$T = T - 1$
14	$n = n + 1$
15	建立第 n 轮分类
16	else
17	$m = m + 1$
18	end while
19	输出：分类 $Q_1(0), Q_2(0), \dots, Q_T(n)$

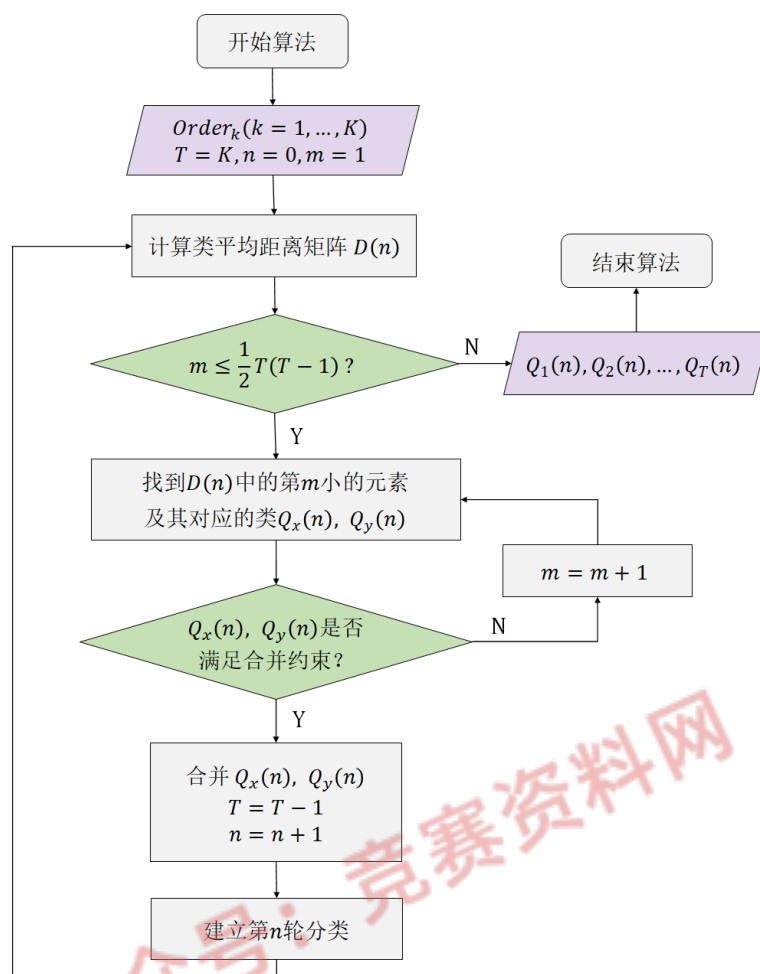


图 11 问题二算法流程图

4.3.4 算法分析

基于类平均距离的凝聚层次聚类算法的主要策略为：将当前聚类结果中满足合并约束的、距离最近的两个类（订单簇）合并。该算法计算复杂度为 $O(n^2)$ ，其嵌套主要体现在类平均距离矩阵 $D(n)$ 的计算与最小元素的遍历中。

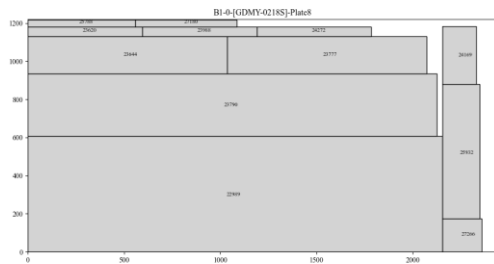
问题二的计算平台与问题一相同。针对问题二的输入数据集 B1、B2、B3、B4、B5，算法分别在用时 **96.54s**、**46.36 s**、**47.80 s**、**43.22 s**、**116.59s** 后完成切割任务，并输出组批结果、排样方案效果图和板材利用率。

4.4 结果分析

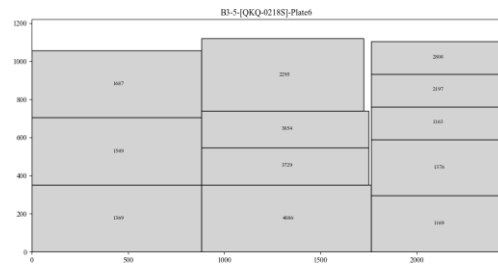
通过上述的基于类平均距离的凝聚层次聚类算法，输入 B 组数据，计算得到问题二的优化结果如表 8 所示。对应于输入数据集 B1、B2、B3、B4 和 B5，需要的总批次数分别为 43、26、25、28 和 44，所需的最少板材数 Y 分别为 **3753 块**、**2517 块**、**2505 块**、**2597 块**和 **4051 块**，板材利用率 η 分别为 **79.34%**、**76.55%**、**77.20%**、**78.17%**和 **76.27%**。图 12 展示了其中的部分排样方案效果图，完整的排样方案效果图见附录 B。

表 8 问题二优化结果

数据集序号	总批次数	最小所需板材数 Y /块	板材利用率 η
B1	43	3753	79.34%
B2	26	2517	76.55%
B3	25	2505	77.20%
B4	28	2597	78.17%
B5	44	4051	76.27%



(1) B1-批次 0-材质[GDMY-0218S]-Plate8



(2) B3-批次 5-材质[QKQ-0218S]-Plate6

图 12 问题二排样方案效果图 (部分)

5. 模型评价与展望

5.1 创新点

1. 分别建立了订单组批和排样优化问题的混合整数规划模型；
2. 针对排样优化问题，利用列生成的思想，设计了基于贪心的深度优先搜索算法；
3. 针对订单组批问题，引入订单相似度评价指标，设计了基于类平均距离的凝聚层次聚类算法。

5.2 优点

1. 本文建立的模型均为单目标优化模型，相比于多目标优化模型，复杂度大大降低；
2. 本文建立的模型中的整数变量均为 0-1 变量，简洁明了；
3. 本文设计的基于贪心的深度优先搜索算法充分考虑了矩形产品的尺寸特点，能够在有效时间内获得质量较高的切割方案，在实际生产中具有较高的应用价值；
4. 本文设计的基于类平均距离的凝聚层次聚类算法不仅能够实现通过组批有效提高原材料利用率，而且结构简洁清晰，具有较强的实际应用性。

5.3 缺点

1. 本文建立的模型中，变量规模随着数据规模的扩大呈爆炸式增长，因此难以应用商业求解器或精确算法来求解大规模问题。
2. 如果需要加工的产品数量巨大，基于贪心的深度优先搜索算法中逐个处理板件的方式可能会造成计算时间过长的问题。后续可以考虑结合并行计算方法来加以改进。
3. 如果需要处理的订单数目非常庞大、材质类型极其繁杂，基于类平均距离的凝聚层次聚类算法中“每个订单自成一类”的初始化方式可能会带来极大的计算复杂度。后续可以考虑在聚类前对订单进行预分类，提升实际计算效率。

5.4 展望

在实际的板材切割问题中，除了需要考虑节约板材原片的成本，还要考虑加工的时间成本以及机器刀口的老化磨损情况，因此除了要提高板材原片的利用率，还要考虑尽可能减少切割次数，以节约加工时间，延缓刀口磨损。而本题中假设每阶段的切割刀数不受限制，后续研究中可以考虑联合优化板材利用率和切割刀数。

在订单组批问题中，本文假设所有订单的交货日期一致，因此不需要考虑订单完成的优先级顺序，使得模型大大简化。但在实际工程应用中，订单交货日期往往是组批时需要考虑的重要因素之一，因此后续可以考虑订单交货日期的影响，设计更加切合实际生产的模型与算法。

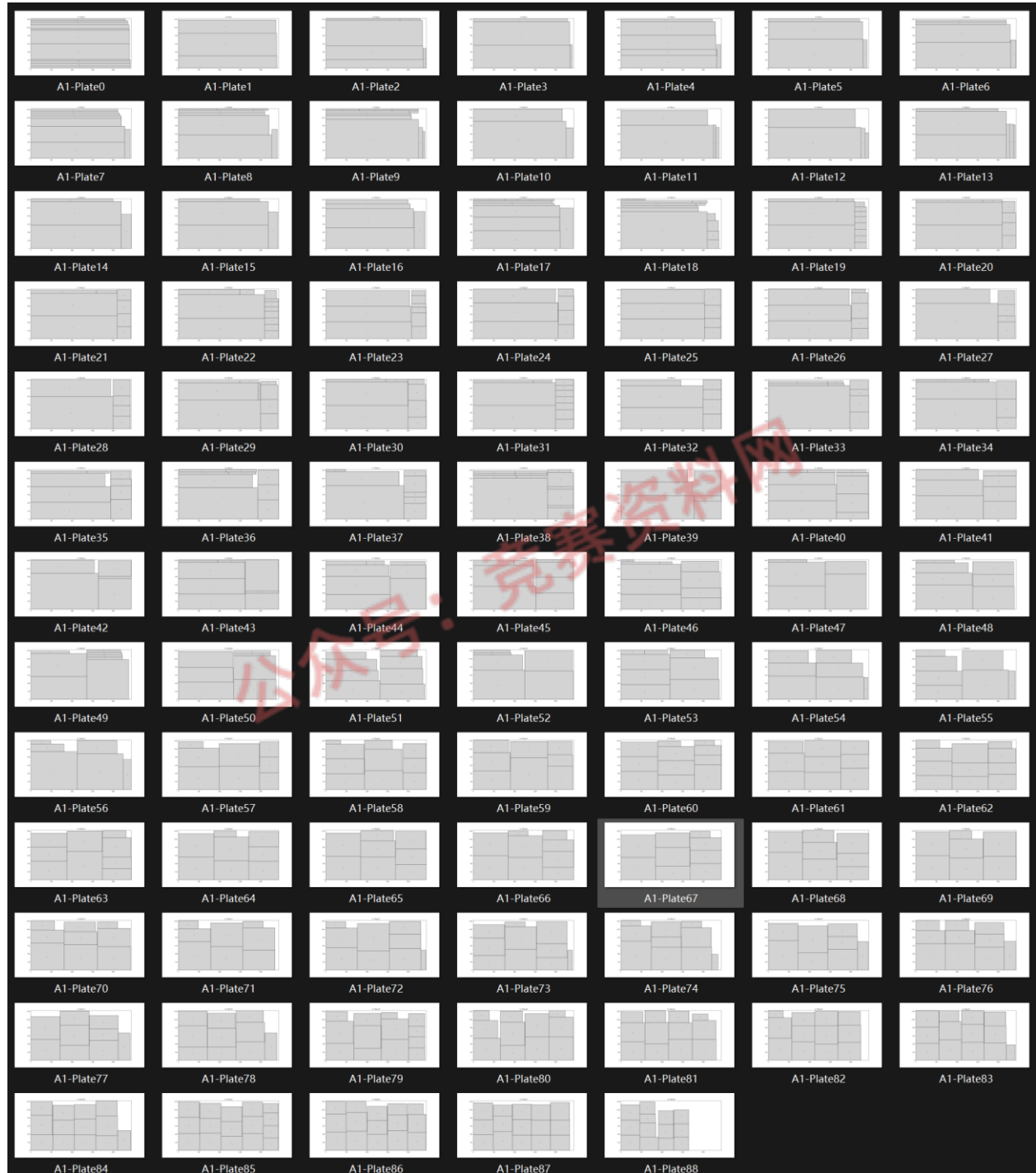
参考文献

- [1] Garey M R, Johnson D S. Computer and Interactability: A Guide of Theory of NP-Completeness. SanFrancisco:Freeman and Company, 1979.
- [2] 黄丹妮.启发式算法求解二维矩形切割优化问题研究[D].华中科技大学, 2019.
- [3] 饶昊.板材切割问题的求解与应用[D].江西财经大学,2019.
- [4] 赵晓东.矩形件优化排样算法的研究与实现[D].大连交通大学, 2008.
- [5] 何霖.满足“一刀切”约束的卷型材矩形件排样方法研究[[D].广东工业大学, 2014.
- [6] Dolatabadi M, Lodi A, Monaci M. Exact algorithms for the two-dimensional guillotine knapsack[J]. Computers & Operations Research, 2012, 39(1):48-53.
- [7] Ahn, S, Yoon K, Park C. An improved best-first branch and bound algorithm for the pallet-loading problem using a staircase structure[J]. Expert Systems with Applications, 2015, 42:7676-7683.
- [8] Furini F, Malaguti E, Duran R M, et al. A column generation heuristic for the two-dimensional two-staged guillotine cutting stock problem with multiple stock size[J]. European Journal of Operational Research, 2012, 218(1):251-260.
- [9] Beyaz M, Dokeroglu T, Cosar A. Robust hyperheuristic algorithms for the offline oriented/nonoriented 2D bin packing problems[J]. Applied Soft Computing, 2015, 36: 236-245.
- [10] Cagan J, Shimada K, Yin S. A survey of computational approaches to three-dimensional layout problems[J]. Computer-Aided Design, 2002, 34(8):597-611.
- [11] Lodi, A., Martello, S. & Vigo, D. Models and Bounds for Two-Dimensional Level Packing Problems. Journal of Combinatorial Optimization 8, 363–379 (2004).
- [12] 张浩.面向板式产品定制生产的组批与排样协同优化方法[D].广东工业大学, 2019.
- [13] 张磊.PCB 定制生产的组批与拼版联合优化方法研究[D].广东工业大学,2019.
- [14] 陈冲.基于拟人一动态规划的板材切割算法研究[D].江西财经大学,2020.
- [15] Parreño, Francisco and Ramón Alvarez-Valdés. Mathematical models for a cutting problem in the glass manufacturing industry. Omega-international Journal of Management Science (2021): 102432.

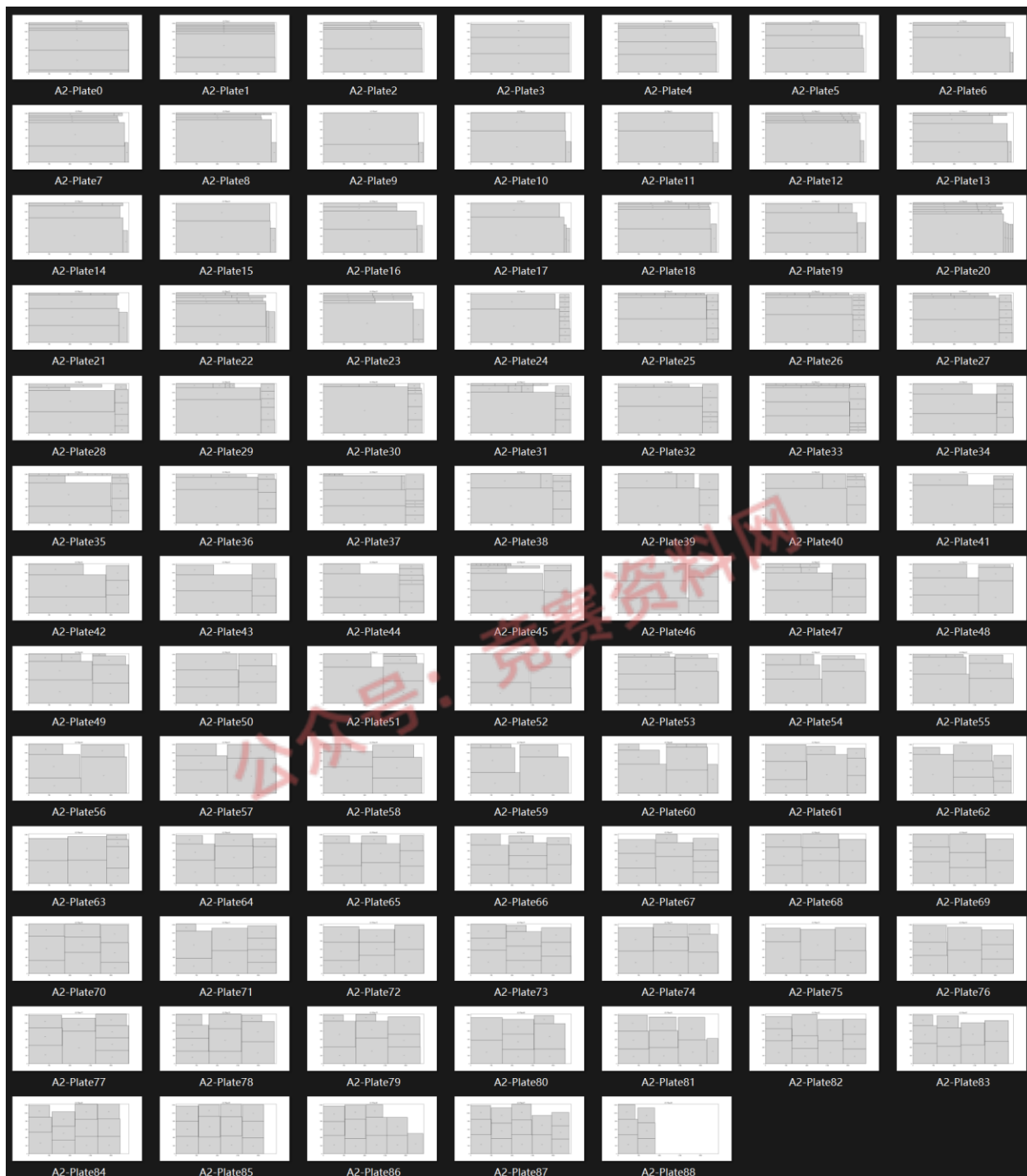
附录

附录 A: 问题一排样方案效果图

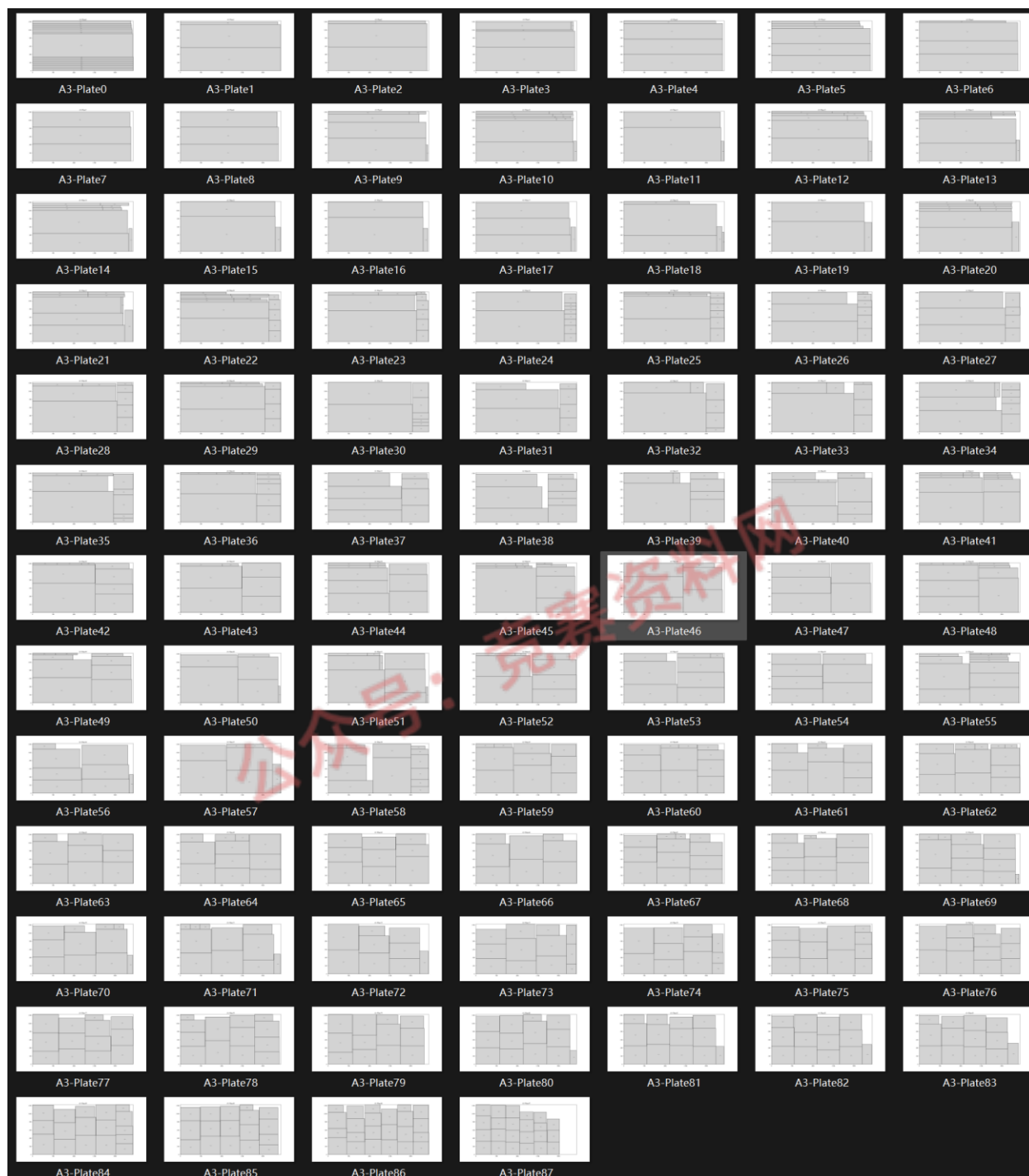
a) dataA1: 89 块原片



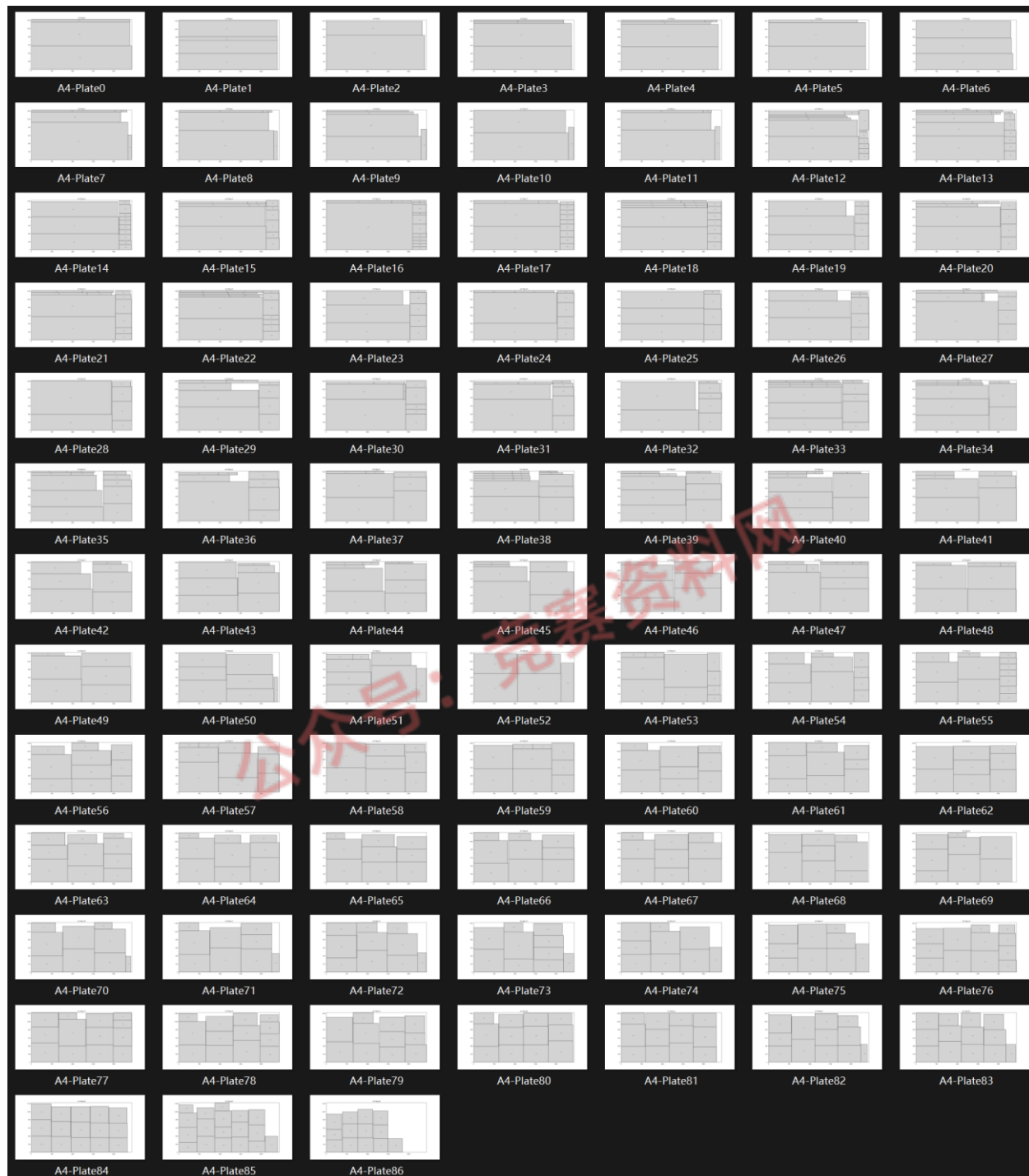
b) dataA2: 89 块原片



c) dataA3: : 88 块原片



d) dataA4: : 87 块原片



附录 B：问题二排样方案效果图

a) dataB1：3753 块原片















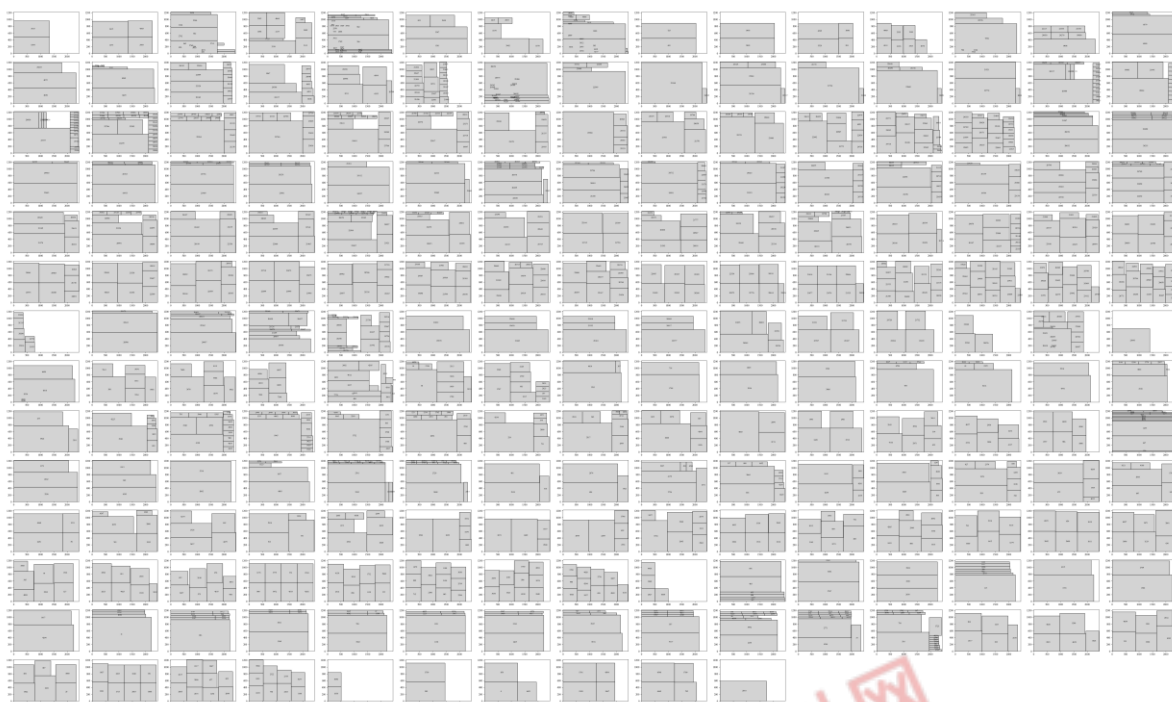
b) dataB2: 2517 块原片



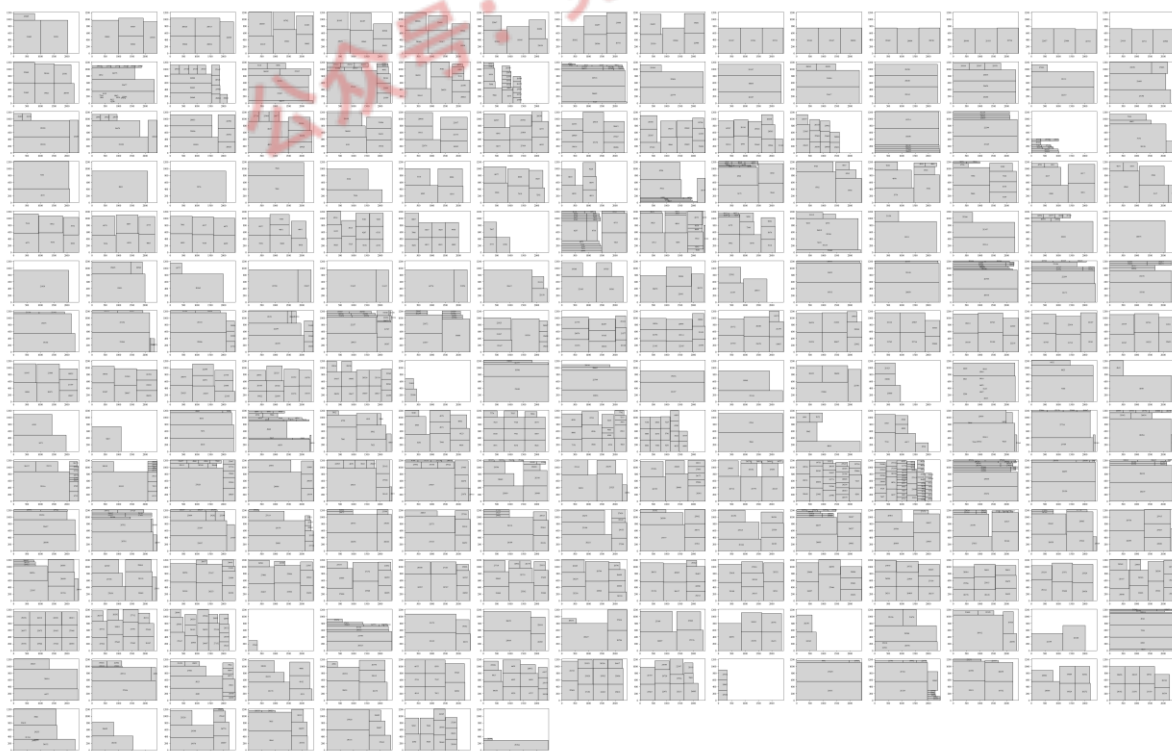








c) dataB3: 2505 块原片











d) dataB4: 2597 块原片



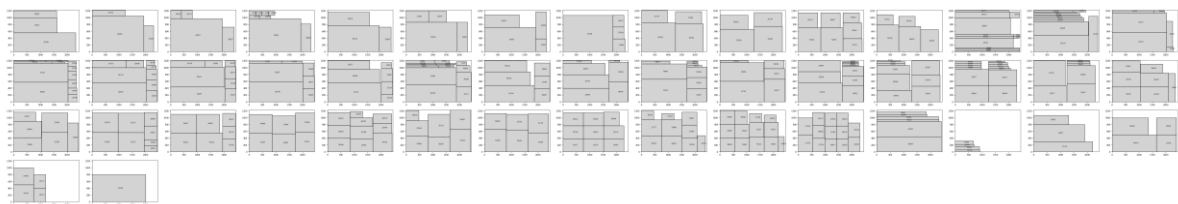








e) dataB5: 4051 块原片

















附录 C: 程序主要模块及数据结构介绍

1. p1.py

(a) 主要类:

- 1) **item**: 产品项 (item) 类, 成员包含序号 **id**、材质 **mat**、长度 **li**、宽度 **wi**、订单号 **order**、左底点横坐标 **x**、左底点纵坐标 **y**、所在原片序号 **plt**、是否转置 **r**
- 2) **stack**: 栈 (stack) 类, 成员包含所含产品项 **items**、长度 **l_**、宽度 **w_**、左底点横坐标 **x**、左底点纵坐标 **y**、所在原片序号 **plt**、是否转置 **r**
重要成员函数: **update_xys(self)**, 用于更新所含 **item** 的位置信息
- 3) **stripe**: 条带 (stripe) 类, 成员包含所含栈 **stacks**、长度 **L**、宽度 **W**、左底点横坐标 **x**、左底点纵坐标 **y**、所在原片序号 **plt**、是否转置 **r**
重要成员函数: **update_xys(self)**, 用于更新所含 **stack** 的位置信息
- 4) **waste**: 边角料类, 成员包含长度 **ll**、宽度 **ww**、左底点横坐标 **x**、左底点纵坐标 **y**、所在原片序号 **plt**

(b) 主要模块

- 1) **load_data(Root, filenum)**: 加载数据, 创建 **item** 对象
- 2) **step1(Stacks)**: 完成长度 $\geq 1220\text{mm}$ 的 **stripe** 构造, 并生成相应的边角料
- 3) **step2(Wastes, Stacks, plt)**: 依次向当前最宽的边角料中放入剩余的 **item**, 若边角料用尽则添加新的原料板
- 4) **show_result(Strips, num)**: 输出排样方案

(c) 数据结构和算法

程序主要采用一维列表存储 **item**、**stack**、**strip**、**waste** 等数据, **stack** 类及 **stripe** 类内采用一维列表嵌套存储所含次级类。各 **strip** 和 **stack** 的构造采用基于贪心的深度优先搜索算法实现。

2. p2.py

(d) 主要类:

- 1) **order**: 订单 (order) 类, 成员包含所含产品项 **items**、订单号 **order_name**、所含材质 **mats**、所含产品项总数 **item_num**、所含产品项总面积 **item_num**
- 2) **cluster**: 批次类, 成员包含所含订单 **orders**、所含材质 **Mats**、所含产品项总数 **item_num**、所含产品项总面积 **item_num**
重要成员函数: **cut(self)**, 用于对当前批次类内的产品项进行排样规划, 具体算法与 **p1.py** 类似

(e) 主要模块

- 1) **load_data(Root, filenum)**: 加载数据, 创建 **order** 对象
- 2) **order_dis(Orders)**: 计算各订单间的相似距离, 以二维数组存储
- 3) **dis_fun(Clu1, Clu2, H)**: 计算两各批次间的平均相似距离, 以判断是否进行聚类
- 4) **Clustering(Clusters, H)**: 聚类操作, 以平均相似距离为对象, 在满足产能约束和生产效率约束的条件下对相似批次进行聚类
- 5) **show_result_cluster**: 输出组批方案

(f) 数据结构和算法

程序主要采用一维列表存储 **order**、**cluster** 等数据, **order** 类、**cluster** 类内采用一维集合存储所含材质, **cluster** 类内采用一维列表嵌套存储所含次级类, 采用二维数组存储各订单间的相似距离。聚类算法通过动态维护一维 **cluster** 数组实现。

附录 D: 程序代码

1. p1.py

```
2. import pandas as pd
3. import os
4. import csv
5. import matplotlib.pyplot as Plt
6. import shutil
7.
8. MAX_L = 2440 #原料长
9. MAX_W = 1220#原料宽
10.
11. class item:#item 类
12.     def __init__(self, data):
13.         super(item, self).__init__()
14.
15.         item_id, mat, li, wi, order = data[0], data[1], data[3], data[4],
            data[5]
16.         self.id = item_id
17.         self.mat = mat
18.         if li >= wi:
19.             self.li = li
20.             self.wi = wi
21.         else:
22.             self.li = wi
23.             self.wi = li
24.         self.order = order
25.         self.x = -1
26.         self.y = -1
27.         self.plt = -1
28.         self.r = -1 #0: 横放, 1: 竖放
29.
30.     def set_x_y(self, x, y):
31.         if self.x == -1:
32.             self.x = x
33.         if self.y == -1:
34.             self.y = y
35.
36.     def set_plt(self, plt):
37.         if self.plt == -1:
38.             self.plt = plt
39.
40.     def set_r(self, r):
41.         if self.r == -1:
42.             self.r = r
```

```

43.
44. class stack:#stack 类
45.     def __init__(self, ini_item):
46.         super(stack, self).__init__()
47.
48.         self.items = [ini_item]
49.         self.w_ = ini_item.wi
50.         self.l_ = ini_item.li
51.         self.x = -1
52.         self.y = -1
53.         self.plt = -1
54.         self.r = -1
55.
56.     def add_item(self, a_item):
57.         self.items.append(a_item)
58.         self.l_ += a_item.li
59.
60.     def set_x_y(self, x, y):
61.         self.x = x
62.         self.y = y
63.
64.     def set_plt(self, plt):
65.         self.plt = plt
66.
67.     def set_r(self, r):
68.         self.r = r
69.
70.     def update_xys(self):#更新所含 item 的位置信息
71.         if self.r == 0:
72.             l_temp = self.x
73.         else:
74.             l_temp = self.y
75.
76.         for num in range(len(self.items)):
77.             self.items[num].set_plt(self.plt)
78.             if self.r == 1:
79.                 self.items[num].set_x_y(self.x, l_temp)
80.             else:
81.                 self.items[num].set_x_y(l_temp, self.y)
82.                 self.items[num].set_r(self.r)
83.                 l_temp += self.items[num].li * (1 - self.items[num].r ^ self.r) +
self.items[num].wi * (self.items[num].r ^ self.r)
84.

```

```

85. class strip:#stripe 类
86.     def __init__(self, ini_stack):
87.         super(strip, self).__init__()
88.
89.         self.stacks = [ini_stack]
90.         self.L = ini_stack.l_
91.         self.W = ini_stack.w_
92.         self.x = -1
93.         self.y = -1
94.         self.plt = -1
95.         self.r = -1
96.
97.     def add_stack(self, a_stack):
98.         self.stacks.append(a_stack)
99.         self.W += a_stack.w_
100.
101.     def set_x_y(self, x, y):
102.         self.x = x
103.         self.y = y
104.
105.     def set_plt(self, plt):
106.         self.plt = plt
107.
108.     def set_r(self, r):
109.         self.r = r #0 横 1 竖
110.
111.     def update_xys(self):#更新所含 stack 的位置信息
112.         if self.r == 0:
113.             w_temp = self.y
114.         else:
115.             w_temp = self.x
116.
117.         for num in range(len(self.stacks)):
118.             self.stacks[num].set_plt(self.plt)
119.             if self.r == 0:
120.                 self.stacks[num].set_x_y(self.x, w_temp)
121.             else:
122.                 self.stacks[num].set_x_y(w_temp, self.y)
123.             self.stacks[num].set_r(self.r)
124.             self.stacks[num].update_xys()
125.             w_temp += self.stacks[num].w_
126.
127.     class waste: #边角料类
128.         def __init__(self, x, y, ww, plt):

```

```

129.         super(waste, self).__init__()
130.         self.x = x
131.         self.y = y
132.         self.ww = ww
133.         self.ll = MAX_W
134.         self.plt = plt
135.
136.     def load_data(Root, filenum):#加载数据
137.         a = pd.read_csv(Root, skiprows=0)
138.         a = a.values
139.         Items = []#item 集合
140.         for row in range(a.shape[0]):
141.             for num in range(a[row][2]):
142.                 a_item = item(a[row])
143.                 Items.append(a_item)
144.             #wi 为第二关键字, li 为第一关键字, 从大到小
145.             Items.sort(key=lambda x: x.wi)
146.             Items.sort(key=lambda x: x.li)
147.             Items = Items[::-1]
148.
149.         pre_observe(Items, filenum)
150.         return Items
151.
152.     def pre_observe(Items, filenum): #观察边长分布
153.         Items_temp = Items.copy()
154.         axis_num_l = []
155.         axis_num_w = []
156.         axis_l = []
157.         axis_w = []
158.         for Item in Items_temp:
159.             if not (Item.li in axis_l):
160.                 axis_num_l.append(1)
161.                 axis_l.append(Item.li)
162.             else:
163.                 axis_num_l[-1] += 1
164.         Items_temp.sort(key=lambda x: x.wi)
165.         for Item in Items_temp:
166.             if not (Item.wi in axis_w):
167.                 axis_num_w.append(1)
168.                 axis_w.append(Item.wi)
169.             else:
170.                 axis_num_w[-1] += 1
171.         Plt.figure(dpi=300, figsize=(10, 6))

```

```

172.     Plt.rc('font', family='SimHei') # 设置中文字体大小和样式
173.     Plt.title("A%d-长度分布柱状图" % filenum)
174.     Plt.grid(axis='y')
175.     Plt.xlabel("边长/mm")
176.     Plt.ylabel('数量/个')
177.     Plt.xlim(0, 2440)
178.     Plt.ylim(0, 1 + max(max(axis_num_l), max(axis_num_w)))
179.     Plt.bar(axis_l, axis_num_l, width=3, align='center', color='#2E75B6')
180.     Plt.savefig("A%d-长度分布柱状图.png" % filenum)
181.
182.     Plt.figure(dpi=300, figsize=(10, 6))
183.     Plt.rc('font', family='SimHei') # 设置中文字体大小和样式
184.     Plt.title("A%d-宽度分布柱状图" % filenum)
185.     Plt.xlabel("边长/mm")
186.     Plt.ylabel('数量/个')
187.     Plt.grid(axis='y')
188.     Plt.xlim(0, 2440)
189.     Plt.ylim(0, 1 + max(max(axis_num_l), max(axis_num_w)))
190.     Plt.bar(axis_w, axis_num_w, width=3, align='center', color='#2E75B6')
191.     Plt.savefig("A%d-宽度分布柱状图.png" % filenum)
192.
193. def get_stacks(Items): #横向拼接 stacks
194.     Stacks = []
195.     for Item in Items:
196.         s = stack(Item)
197.         Stacks.append(s)
198.     #w_为第二关键字, l_为第一关键字, 从大到小
199.     Stacks.sort(key=lambda x: x.w_)
200.     Stacks.sort(key=lambda x: x.l_)
201.     Stacks = Stacks[::-1]
202.     return Stacks
203.
204. #拼出大 strip
205. def step1(Stacks):
206.     plt = -1
207.     Strips = []
208.     Wastes = []
209.     while True:
210.         cut_num = []
211.         # 先看横放
212.         for num in range(len(Stacks)):
213.             if num in cut_num:
214.                 continue

```

```

215.         # 长度满足
216.         if len(cut_num) == 0:
217.             plt += 1
218.             cut_num.append(num)
219.             S = strip(Stacks[num])
220.             S.set_plt(plt)
221.             S.set_r(0)
222.             S.set_x_y(0, 0)
223.             Strips.append(S)
224.         else:
225.             if Stacks[num].w_ + Strips[-1].W <= MAX_W: # 宽度满足
226.                 Strips[-1].add_stack(Stacks[num]) # 向上拼接
227.                 cut_num.append(num)
228.                 for stack_num in range(num + 1, len(Stacks)):
229.                     if stack_num in cut_num:
230.                         continue
231.                     #横着拼右边
232.                     if Stacks[stack_num].w_ == Strips[-1].stacks[-1].w_
233. and Stacks[stack_num].l_ + Strips[-1].stacks[-1].l_ <= Strips[-1].L:
234.                         for Item in Stacks[stack_num].items:
235.                             Strips[-1].stacks[-1].add_item(Item)
236.                             cut_num.append(stack_num)
237.                     # 竖着拼右边
238.                     for stack_num in range(num + 1, len(Stacks)):
239.                         if stack_num in cut_num:
240.                             continue
241.                         if Stacks[stack_num].l_ == Strips[-1].stacks[-1].w_
242. and Stacks[stack_num].w_ + Strips[-1].stacks[-1].l_ <= Strips[-1].L:
243.                             Stacks[stack_num].items[0].set_r(1)
244.                             Strips[-1].stacks[-
245. 1].add_item(Stacks[stack_num].items[0])
246.                             cut_num.append(stack_num)
247.             if len(cut_num) > 0: # 横放成功
248.                 Strips[-1].update_xys()
249.                 new_waste = waste(0 + Strips[-1].L, 0, MAX_L - Strips[-1].L,
250. plt)
251.                 # 横放右侧产生新的边角料
252.                 Wastes.append(new_waste)
253.                 Wastes.sort(key=lambda x: x.ww)
254.                 Wastes = Wastes[::-1]
255.                 # 剪掉用过的 stack
256.                 temp_Stacks = []
257.                 for num in range(len(Stacks)):
258.                     if not (num in cut_num):

```

```

256.             temp_Stacks.append(Stacks[num])
257.             Stacks = temp_Stacks
258.
259.             if len(Stacks) == 0 or (len(Stacks) > 0 and Stacks[0].l_ <=
MAX_W):
260.                 break
261.
262.             return Strips, Wastes, Stacks, plt
263.
264.     #塞入剩余 stack
265.     def step2(Wastes, Stacks, plt):
266.         Strips = []
267.
268.         while len(Stacks) > 0:
269.             while len(Wastes) > 0:
270.                 a_waste = Wastes[0]
271.                 cut_num = []
272.                 #取出边角料
273.                 if len(Wastes) > 1:
274.                     Wastes = Wastes[1:]
275.                 else:
276.                     Wastes = []
277.                 # 先看横放
278.                 for num in range(len(Stacks)):
279.                     if Stacks[num].l_ > a_waste.w_ or (num in cut_num):
280.                         continue
281.                     #长度满足
282.                     if len(cut_num) == 0:
283.                         cut_num.append(num)
284.                         S = strip(Stacks[num])
285.                         S.set_plt(a_waste.plt)
286.                         S.set_r(0)
287.                         S.set_x_y(a_waste.x, a_waste.y)
288.                         Strips.append(S)
289.                     else:
290.                         if Stacks[num].w_ + Strips[-1].W <= MAX_W:#宽度满足
291.                             Strips[-1].add_stack(Stacks[num]) # 向上拼接
292.                             cut_num.append(num)
293.                         #横着拼右边
294.                         for stack_num in range(num + 1, len(Stacks)):
295.                             if stack_num in cut_num:
296.                                 continue
297.                             if Stacks[stack_num].w_ == Strips[-1].stacks[-
1].w_ and Stacks[stack_num].l_ + Strips[-1].stacks[-1].l_ <= Strips[-1].L:

```

```

298.         for Item in Stacks[stack_num].items:
299.             Strips[-1].stacks[-1].add_item(Item)
300.             cut_num.append(stack_num)
301.         #竖着拼右边
302.         for stack_num in range(num + 1, len(Stacks)):
303.             if stack_num in cut_num:
304.                 continue
305.             if Stacks[stack_num].l_ == Strips[-1].stacks[-1].w_ and Stacks[stack_num].w_ + Strips[-1].stacks[-1].l_ <= Strips[-1].L and len(Stacks[stack_num].items) == 1:
306.                 Stacks[stack_num].items[0].set_r(1)
307.                 Strips[-1].stacks[-1].add_item(Stacks[stack_num].items[0])
308.                 cut_num.append(stack_num)
309.
310.         if len(cut_num) > 0: #横放成功
311.             # 剪掉用过的 stack
312.             temp_Stacks = []
313.             for num in range(len(Stacks)):
314.                 if not (num in cut_num):
315.                     temp_Stacks.append(Stacks[num])
316.             Stacks = temp_Stacks
317.             #如果横 item 已用尽, 考虑竖放 item
318.             if len(Stacks) > 0 and (Stacks[-1].l_ + Strips[-1].W <=
MAX_W):
319.                 Stacks_vec = Stacks.copy()
320.                 cut_num_vec = []
321.                 Stacks_vec.sort(key=lambda x: x.l_)
322.                 Stacks_vec.sort(key=lambda x: x.w_)
323.                 Stacks_vec = Stacks_vec[::-1]
324.                 for num in range(len(Stacks_vec)):
325.                     if Stacks_vec[num].l_ + Strips[-1].W > MAX_W or
(num in cut_num_vec):
326.                         continue
327.                     #长度满足
328.                     if Stacks_vec[num].w_ <= Strips[-1].L: #宽度满足
329.                         cut_num_vec.append(num)
330.                         Stacks_vec[num].items[0].set_r(1)
331.                         temp = Stacks_vec[num].w_
332.                         Stacks_vec[num].w_ = Stacks_vec[num].l_
333.                         Stacks_vec[num].l_ = temp
334.                         Strips[-1].add_stack(Stacks_vec[num])
335.                         for stack_vec_num in range(num + 1,
len(Stacks_vec)):
336.                             if stack_vec_num in cut_num_vec:

```

```

337.                 continue
338.                 if Stacks_vec[stack_vec_num].l_ == Strips[-1].stacks[-1].w_ and Stacks_vec[stack_vec_num].w_ + Strips[-1].stacks[-1].l_
                    <= Strips[-1].L:
339.                     Stacks_vec[num].items[0].set_r(1)
340.                     Strips[-1].stacks[-1].add_item(Stacks_vec[stack_vec_num].items[0])
341.                     cut_num_vec.append(stack_vec_num)
342.                     # 剪掉用过的 stack_vec
343.                     temp_Stacks_vec = []
344.                     for num in range(len(Stacks_vec)):
345.                         if not (num in cut_num_vec):
346.                             temp_Stacks_vec.append(Stacks_vec[num])
347.                     Stacks_vec = temp_Stacks_vec
348.                     Stacks_vec.sort(key=lambda x: x.w_)
349.                     Stacks_vec.sort(key=lambda x: x.l_)
350.                     Stacks = Stacks_vec[::-1]
351.
352.                     Strips[-1].update_xys()
353.                     new_waste = waste(a_waste.x + Strips[-1].L, a_waste.y,
                    a_waste.ww - Strips[-1].L, a_waste.plt)
354.                     #横放右侧产生新的边角料
355.                     Wastes.append(new_waste)
356.                     Wastes.sort(key=lambda x: x.ww)
357.                     Wastes = Wastes[::-1]
358.
359.                 else:
360.                     #再看竖放:
361.                     for num in range(len(Stacks)):
362.                         if Stacks[num].w_ > a_waste.ww or (num in cut_num) or
                    Stacks[num].l_ > MAX_W:
363.                             continue
364.                             # 宽度满足
365.                             if len(cut_num) == 0:
366.                                 cut_num.append(num)
367.                                 S = strip(Stacks[num])
368.                                 S.set_plt(a_waste.plt)
369.                                 S.set_r(1)
370.                                 S.set_x_y(a_waste.x, a_waste.y)
371.                                 Strips.append(S)
372.                             else:
373.                                 if Stacks[num].w_ + Strips[-1].W <= a_waste.ww: #
                    宽度满足
374.                                     Strips[-1].add_stack(Stacks[num]) # 向右拼接
375.                                     cut_num.append(num)

```

```

376.         for stack_num in range(num + 1, len(Stacks)):
377.             if stack_num in cut_num:
378.                 continue
379.             if Stacks[stack_num].w_ == Strips[-1].stacks[-
1].w_ and Stacks[stack_num].l_ + Strips[-1].stacks[-1].l_ <= Strips[-1].L:
380.                 for Item in Stacks[stack_num].items:
381.                     Strips[-1].stacks[-1].add_item(Item)
382.                     cut_num.append(stack_num)
383.             for stack_num in range(num + 1, len(Stacks)):
384.                 if stack_num in cut_num:
385.                     continue
386.                 if Stacks[stack_num].l_ == Strips[-1].stacks[-
1].w_ and Stacks[stack_num].w_ + Strips[-1].stacks[-1].l_ <= Strips[-1].L and
len(Stacks[stack_num].items) == 1:
387.                     Stacks[stack_num].Items[0].set_r(0)
388.                     Strips[-1].stacks[-
1].add_item(Stacks[stack_num].Items[0])
389.                     cut_num.append(stack_num)
390.                 if len(cut_num) > 0: # 竖放成功
391.                     Strips[-1].update_xys()
392.                     # 剪掉用过的 stack
393.                     temp_Stacks = []
394.                     for num in range(len(Stacks)):
395.                         if not (num in cut_num):
396.                             temp_Stacks.append(Stacks[num])
397.                     Stacks = temp_Stacks
398.
399.             if len(Stacks) > 0: # 遍历完 waste, 但还有 stack 剩余
400.                 plt += 1
401.                 new_waste = waste(0, 0, MAX_L, plt) # 新原料板, 相当于大的边角料
402.                 Wastes.append(new_waste)
403.         return Strips, plt
404.
405.     def show_result(Strips, num): # 输出方案
406.         s = 0
407.         data = []
408.         for Strip in Strips:
409.             for Stack in Strip.stacks:
410.                 for Item in Stack.items:
411.                     s += Item.li * Item.wi # 统计面积
412.                     data.append([Item.mat, Item.plt, Item.id, Item.x, Item.y,
(1 - Item.r) * Item.li + Item.r * Item.wi, Item.r * Item.li + (1 - Item.r) *
Item.wi])
413.         # 按原料板编号排序

```

```

414.     data.sort(key=lambda x: x[1])
415.
416.     with open('cut_program%d.csv' % num, mode='w', newline='') as f:
417.         writer = csv.writer(f)
418.         writer.writerow(['原片材质', '原片序号', '产品 id', '产品 x 坐标', '产
    品 y 坐标', '产品 x 方向长度', '产品 y 方向长度'])
419.         for row in data:
420.             writer.writerow(row)
421.
422.     plot(data, num)#绘制排版方案
423.     return s
424.
425. def plot(data, num):#绘制排版方案
426.     if not os.path.exists('figures%s' % num):
427.         os.mkdir('figures%s' % num)
428.     else:
429.         shutil.rmtree('figures%s' % num)
430.         os.mkdir('figures%s' % num)
431.     plt_temp = 0
432.     fig1 = Plt.figure(dpi=300, figsize=(12, 6))
433.     Plt.rc('font', family='Times New Roman')
434.     ax = fig1.add_subplot(111)
435.     Plt.title('A%d' % num + '-Plate%d' % plt_temp)
436.     ax.set_xlim(0, 2440)
437.     ax.set_ylim(0, 1220)
438.     for row in range(len(data)):
439.         if data[row][1] == plt_temp:
440.             #item 图形
441.             rect = Plt.Rectangle((data[row][3], data[row][4]),
    data[row][5], data[row][6], linewidth=1, edgecolor='black',
    facecolor='#D3D3D3')
442.             #item 编号
443.             Plt.text(data[row][3] + 0.48 * data[row][5], data[row][4] +
    0.48 * data[row][6], str(data[row][2]), fontsize=8, family='Times New Roman')
444.             ax.add_patch(rect)
445.         else:
446.             # Plt.show()
447.             Plt.savefig(os.path.join('figures%s' % num, 'A%d' % num + '-
    Plate%d.png' % plt_temp))
448.             plt_temp = data[row][1]
449.             fig1 = Plt.figure(dpi=300, figsize=(12, 6))
450.             ax = fig1.add_subplot(111)
451.             ax.set_xlim(0, 2440)
452.             ax.set_ylim(0, 1220)

```

```

453.         Plt.rc('font', family='Times New Roman')
454.         Plt.title('A%d' % num + '-Plate%d' % plt_temp)
455.         rect = Plt.Rectangle((data[row][3], data[row][4]),
            data[row][5], data[row][6], linewidth=1, edgecolor='black',
            facecolor='#D3D3D3')
456.         Plt.text(data[row][3] + 0.48 * data[row][5], data[row][4] +
            0.48 * data[row][6], str(data[row][2]), fontsize=8, family='Times New Roman')
457.         ax.add_patch(rect)
458.         if row == len(data) - 1:
459.             Plt.savefig(os.path.join('figures%s' % num, 'A%d' % num + '-
                Plate%d.png' % plt_temp))
460.
461.     if __name__ == '__main__':
462.         for i in range(1, 5):
463.             root = 'dataA%d.csv' % i
464.             items = load_data(root, i)
465.
466.             # 按wi 拼接
467.             stacks = get_stacks(items)
468.             #拼出大块 stripe
469.             strips, wastes, stacks_remaining, plt0 = step1(stacks)
470.             #塞入剩余 stack
471.             strips_add, plt1 = step2(wastes, stacks_remaining, plt0)
472.             #所需原料板数
473.             print(plt1 + 1)
474.             strips.extend(strips_add)
475.             #输出坐标和利用率
476.             s_total = show_result(strips, i)
477.             rate_real = s_total / (MAX_L * MAX_W * (plt1 + 1))
478.             print(rate_real)

```

2. p2.py

```

3. from p1 import item, step1, step2, get_stacks
4. import pandas as pd
5. import os
6. import csv
7. import matplotlib.pyplot as Plt
8. import shutil
9. import numpy as np
10.
11. MAX_ITEM_NUM = 1000
12. MAX_ITEM_AREA = 250 * 1e+6
13. MAX_W = 1220
14. MAX_L = 2440

```

```

15.
16. class order: #订单类
17.     def __init__(self, ini_item):
18.         super(order, self).__init__()
19.         self.items = [ini_item]
20.         self.order_name = ini_item.order
21.         self.mats = {ini_item.mat}
22.         self.item_area = ini_item.li * ini_item.wi
23.         self.item_num = 1
24.
25.     def update(self, a_item):
26.         self.items.append(a_item)
27.         self.mats.update(a_item.mat)
28.         self.item_num += 1
29.         self.item_area += a_item.li * a_item.wi
30.
31. class cluster: #组批
32.     def __init__(self, ini_order):
33.         super(cluster, self).__init__()
34.         self.orders = [ini_order]
35.         self.item_num = ini_order.item_num
36.         self.item_area = ini_order.item_area
37.         self.Mats = ini_order.mats
38.
39.     def update_(self, a_cluster):
40.         self.orders.extend(a_cluster.orders)
41.         self.item_num += a_cluster.item_num
42.         self.item_area += a_cluster.item_area
43.         self.Mats.update(a_cluster.Mats)
44.
45.     def cut(self): #切割排版
46.         Items = [] #全部 Item
47.         Strips_all = []
48.         Plt_all = 0
49.         for Order in self.orders:
50.             Items.extend(Order.items)
51.         Items.sort(key=lambda x: x.mat) #按材质排序
52.         Items_same_mat = [[Items[0]]] #[[材质 1], [材质 2], ...]
53.         for item_num in range(1, len(Items)):
54.             if Items[item_num].mat == Items_same_mat[-1][0].mat:
55.                 Items_same_mat[-1].append(Items[item_num])
56.             else:
57.                 Items_same_mat.append([Items[item_num]])

```

```

58.     for list_num in range(len(Items_same_mat)):
59.         items_now = Items_same_mat[list_num]
60.         # wi 为第二关键字, li 为第一关键字, 从大到小
61.         items_now.sort(key=lambda x: x.wi)
62.         items_now.sort(key=lambda x: x.li)
63.         items_now = items_now[::-1]
64.         Stacks = get_stacks(items_now)
65.         Strips, Wastes, Stacks_remaining, Plt0 = step1(Stacks)
66.         Strips_add, Plt1 = step2(Wastes, Stacks_remaining, Plt0)
67.         Strips.extend(Strips_add)
68.         Strips_all.append(Strips)
69.         Plt_all += Plt1 + 1
70.     Rate = self.item_area / (MAX_L * MAX_W * Plt_all)
71.     return Plt_all, Rate, Strips_all
72.
73. def load_data(Root):
74.     a = pd.read_csv(Root, skiprows=0)
75.     a = a.values
76.     Items = [] #item 集合
77.     Orders = [] #order 集合
78.     for row in range(a.shape[0]):
79.         for num in range(a[row][2]):
80.             a_item = item(a[row])
81.             Items.append(a_item)
82.     #order 为关键字, 从大到小
83.     Items.sort(key=lambda x: int(x.order[5:]))
84.     a_order = order(Items[0])
85.     Orders.append(a_order)
86.     for num in range(1, len(Items)):
87.         if Items[num].order == Orders[-1].order_name:
88.             Orders[-1].update(Items[num])
89.         else:
90.             a_order = order(Items[num])
91.             Orders.append(a_order)
92.     return Orders
93.
94. def get_clusters(Orders): #自成一簇
95.     Clusters = []
96.     for Order in Orders:
97.         Clu = cluster(Order)
98.         Clusters.append(Clu)
99.     return Clusters
100.

```

```

101. def order_dis(Orders): #计算每个 order 的相似距离
102.     L = int(Orders[-1].order_name[5:])
103.     H = np.empty([L, L])
104.     for order_num_i in range(len(Orders)):
105.         for order_num_j in range(order_num_i + 1, len(Orders)):
106.             id_i = int(Orders[order_num_i].order_name[5:])-1
107.             id_j = int(Orders[order_num_j].order_name[5:])-1
108.             H[id_i, id_j] = H[id_j, id_i] = 1 -
                len(Orders[order_num_i].mats.intersection(Orders[order_num_j].mats)) /
                len(Orders[order_num_i].mats.union(Orders[order_num_j].mats))
109.
110.     return H
111.
112. def dis_fun(Clu1, Clu2, H): #计算两个簇的相似距离
113.     dis_total = 0
114.     for order1 in Clu1.orders:
115.         for order2 in Clu2.orders:
116.             id_i = int(order1.order_name[5:])-1
117.             id_j = int(order2.order_name[5:])-1
118.             dis_total += H[id_i, id_j]
119.
120.     return dis_total / (len(Clu1.orders) * len(Clu2.orders))
121.
122. def Clustering(Clusters, H):#进行聚类
123.     while True:
124.         min_dis = 32767
125.         min_which = []
126.         for num1 in range(len(Clusters)):
127.             for num2 in range(num1 + 1, len(Clusters)):
128.                 dis_now = dis_fun(Clusters[num1], Clusters[num2], H)
129.                 if dis_now < min_dis and (Clusters[num1].item_num +
                    Clusters[num2].item_num <= MAX_ITEM_NUM) and (Clusters[num1].item_area +
                    Clusters[num2].item_area <= MAX_ITEM_AREA):
130.                     min_dis = dis_now
131.                     min_which = [num1, num2]
132.             print(len(Clusters), min_dis)
133.             if len(min_which) == 0:
134.                 break
135.             temp_Clusters = []
136.             for num in range(len(Clusters)):
137.                 if not (num in min_which):
138.                     temp_Clusters.append(Clusters[num])
139.             Clusters[min_which[0]].update_(Clusters[min_which[1]])

```

```

140.         temp_Clusters.append(Clusters[min_which[0]])
141.         Clusters = temp_Clusters
142.         return Clusters
143.
144.     def show_result_cluster(Cluster_num, Strips, filenum):
145.         data = []
146.         for Strip in Strips:
147.             for Stack in Strip.stacks:
148.                 for Item in Stack.items:
149.                     data.append([Cluster_num, Item.mat, Item.plt, Item.id,
150.                                 Item.x, Item.y, (1 - Item.r) * Item.li + Item.r * Item.wi,
151.                                 Item.r * Item.li + (1 - Item.r) * Item.wi])
152.                     # 按原料板编号为第二关键字，批次号为第一关键字排序
153.                     data.sort(key=lambda x: x[2])
154.                     data.sort(key=lambda x: x[0])
155.                     with open('sum_order%d.csv' % filenum, mode='a', newline='') as f:
156.                         writer = csv.writer(f)
157.                         for row in data:
158.                             writer.writerow(row)
159.
160.                     plot(data, filenum) # 绘制排版方案
161.
162.     def plot(data, filenum):
163.
164.         plt_temp = 0
165.         mat_temp = data[0][1]
166.         cluster_temp = data[0][0]
167.         fig1 = Plt.figure(dpi=100, figsize=(12, 6))
168.         count = 1
169.         Plt.rc('font', family='Times New Roman')
170.         ax = fig1.add_subplot(3, 3, count)
171.         Plt.title('B%d-' % filenum + '%d' % cluster_temp + '-[' + mat_temp +
172.                  ']-Plate%d' % plt_temp)
173.         ax.set_xlim(0, 2440)
174.         ax.set_ylim(0, 1220)
175.         for row in range(len(data)):
176.             if data[row][2] == plt_temp:
177.                 #item 图形
178.                 rect = Plt.Rectangle((data[row][4], data[row][5]),
179.                                     data[row][6], data[row][7], linewidth=1, edgecolor='black',
180.                                     facecolor='#D3D3D3')
181.                 #item 编号

```

```

179.         Plt.text(data[row][4] + 0.48 * data[row][6], data[row][5] +
    0.48 * data[row][7], str(data[row][3]), fontsize=8, family='Times New Roman')
180.         ax.add_patch(rect)
181.     else:
182.         # Plt.show()
183.         # Plt.savefig(os.path.join('figuresB%s' % filenum, 'B%d-' %
    filenum + '%d' % cluster_temp + '-[' + mat_temp + ']-Plate%d.png' %
    plt_temp))
184.         plt_temp = data[row][2]
185.         # Plt.close()
186.         if count == 9:
187.             Plt.savefig(os.path.join('figuresB%s' % filenum, 'B%d-' %
    filenum + '%d' % cluster_temp + '-[' + mat_temp + ']-Plate%d.png' %
    plt_temp))
188.             Plt.close()
189.             fig1 = Plt.figure(dpi=100, figsize=(12, 6))
190.             count = 1
191.         else:
192.             count += 1
193.             ax = fig1.add_subplot(3, 3, count)
194.             ax.set_xlim(0, 2440)
195.             ax.set_ylim(0, 1220)
196.             Plt.rc('font', family='Times New Roman')
197.             Plt.title('B%d-' % filenum + '%d' % cluster_temp + '-[' +
    mat_temp + ']-Plate%d' % plt_temp)
198.             rect = Plt.Rectangle((data[row][4], data[row][5]),
    data[row][6], data[row][7], linewidth=1, edgecolor='black',
    facecolor='#D3D3D3')
199.             Plt.text(data[row][4] + 0.48 * data[row][6], data[row][5] +
    0.48 * data[row][7], str(data[row][3]), fontsize=8, family='Times New Roman')
200.             ax.add_patch(rect)
201.             if row == len(data) - 1:
202.                 Plt.savefig(os.path.join('figuresB%s' % filenum, 'B%d-' %
    filenum + '%d' % cluster_temp + '-[' + mat_temp + ']-Plate%d.png' %
    plt_temp))
203.                 Plt.close()
204.
205.     if __name__ == '__main__':
206.         for i in range(1, 6):
207.             with open('sum_order%d.csv' % i, mode='w', newline='') as F:
208.                 Writer = csv.writer(F)
209.                 Writer.writerow(['批次数号', '原片材质', '原片序号', '产品 id', '
    产品 x 坐标', '产品 y 坐标', '产品 x 方向长度', '产品 y 方向长度'])
210.             if not os.path.exists('figuresB%s' % i):

```

```

211.         os.mkdir('figuresB%s' % i)
212.     else:
213.         shutil.rmtree('figuresB%s' % i)
214.         os.mkdir('figuresB%s' % i)
215.     root = 'dataB%d.csv' % i
216.     orders = load_data(root)
217.     #计算相似距离
218.     H_base = order_dis(orders)
219.
220.     #自成一簇
221.     clusters = get_clusters(orders)
222.
223.     #聚类
224.     clusters_final = Clustering(clusters, H_base)
225.     #开始排版
226.     plt_total = 0
227.     area_total = 0
228.     for cluster_num in range(len(clusters_final)):
229.         plt_one_cluster, rate_one_cluster, strips_one_cluster =
            clusters_final[cluster_num].cut()
230.         plt_total += plt_one_cluster
231.         area_total += clusters_final[cluster_num].item_area
232.         for strips in strips_one_cluster:
233.             show_result_cluster(cluster_num, strips, i)
234.
235.     rate_real = area_total / (MAX_L * MAX_W * plt_total)
236.     print(plt_total)
237.     print(rate_real)

```